



ABSTRACCION Y USO DE DATOS

Instituto Politécnico Nacional



Integrantes del equipo:

Grande Quintana Jorge Daniel

Herrera Quiroz Aolani Tziranda

Verboonen Sierra Edgar Jaret

7 DE ABRIL DE 2026

Secuencia: 3CM30

Reporte 2do Parcial Programas

¿Qué finalidad tiene este reporte que estamos llevando a cabo?

En este reporte documentaremos el código completo de cada programa. Puede que no aparezca todas las capturas de pantalla de los códigos debido a la resolución que se presenta, sin embargo, tenemos la facilidad de poder tenerlo en GitHub y ahí poder visualizarlo mejor y manipular si se crea una duda de “como se hizo”, “que pasa”, Etc.

Nuestra finalidad principal es consolidar los aprendizajes obtenidos en esta etapa, enfocándonos en dos grandes pilares de la computación: **las Estructuras de Datos (Pilas, Colas y Listas)** y los **Algoritmos de Ordenamiento (Burbuja, Merge y Quick Sort)**.

A lo largo de los ejercicios, veremos la evolución de nuestros programas; pasaremos de implementaciones estáticas utilizando arreglos, al reto del manejo de memoria dinámica mediante punteros, perdiéndole el miedo a la complejidad que esto conlleva. Además, contrastaremos la manipulación de datos básicos con la creación y ordenamiento de Nuevos Tipos de Dato mediante el paradigma Orientado a Objetos (POO).

En su mayoría, los programas siguen una progresión natural y van de la mano; es decir, la lógica que construimos en un ejercicio evoluciona en el siguiente para adaptarse a nuevas reglas. Esto nos permite tener un "mapa" de aprendizaje continuo al terminarlos.

Si durante el desarrollo nos enfrentamos a errores lógicos o de memoria, se documentarán junto con su respectiva solución. El análisis no será una explicación exhaustiva de cada línea de código, sino una guía concisa. El contenido de cada apartado incluirá:

- **Objetivo:** Lo que hace el programa y la estructura o algoritmo que utiliza.
- **Implementación:** El código fuente completo junto con la evidencia de su compilación y ejecución.
- **Análisis:** Una explicación breve de la lógica utilizada para comprender qué sucedió detrás de escena.
- **Resolución de problemas:** Si se encontraron errores durante el desarrollo, se mostrarán acompañados de su solución y justificación técnica.

En todos los programas tendremos 3 archivos que estarán fijos, a excepción de nuevo tipo de dato, que en este contendrá un archivo extra. Los 3 archivos mencionado serán:

Implementación

La implementación se basa en una clase abstracta y una clase concreta:

Clase Abstracta

Define la estructura base de la pila, cola y lista incluyendo:

- Métodos abstractos:
 - agregar()
 - quitar()
 - mostrar()
 - estaVacía()
 - estaLlena()
 - tamaño()

Clase Concreta

Implementa la lógica funcional de la pila, cola y lista.

Funcionamiento del Programa

(La interfaz que sería el Main)

El programa cuenta con un menú interactivo que permite al usuario:

1. Agregar elementos a la pila/cola/lista
2. Quitar elementos
3. Mostrar el contenido
4. Verificar si está vacía
5. Verificar si está llena
6. Consultar tamaño

El flujo del programa se ejecuta dentro de un ciclo repetitivo hasta que el usuario decide salir.

==> INDICE <==

HECHOS EN POO

I. Algoritmos de Ordenamiento Directo

- 16. Ordenamiento Burbuja: (Nuevo tipo de dato)
- 17. Ordenamiento Merge Sort: (Nuevo tipo de dato)
- 18. Ordenamiento QuickSort: (Nuevo tipo de dato)

(Dato básico y Nuevo tipo de dato)

ADT (Abstract Data Type)

II. Estructuras de Datos: Arreglos, Punteros, Librerías (PILA)

1. Array

Manejo de una pila de manera estática por medio de un arreglo

2. Punteros

Manejo de una pila de manera dinámica por medio de punteros al mismo tipo de dato de la pila, considerando la referencia

3. Librerías

Manejo de una pila de manera dinámica por medio de las librerías o clases proporcionadas por el IDE.

III. Estructuras de Datos: Arreglos, Punteros, Librerías (COLA)

4. Array

Manejo de una pila de manera estática por medio de un arreglo

5. Punteros

Manejo de una pila de manera dinámica por medio de punteros al mismo tipo de dato de la pila, considerando la referencia

6. Librerías

Manejo de una pila de manera dinámica por medio de las librerías o clases proporcionadas por el IDE.

IV. Estructuras de Datos: Arreglos, Punteros, Librerías (LISTAS)

7. Array

Manejo de una pila de manera estática por medio de un arreglo

8. Punteros

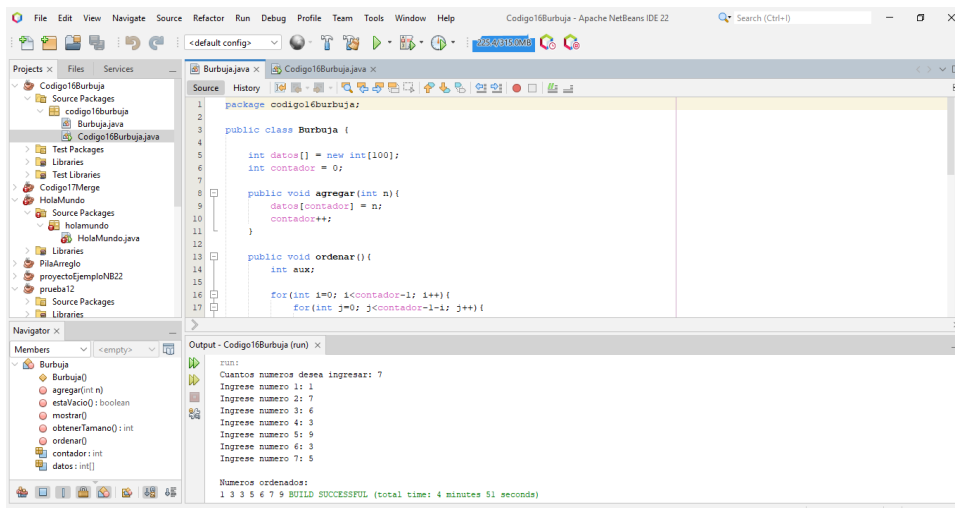
Manejo de una pila de manera dinámica por medio de punteros al mismo tipo de dato de la pila, considerando la referencia

9. Librerías

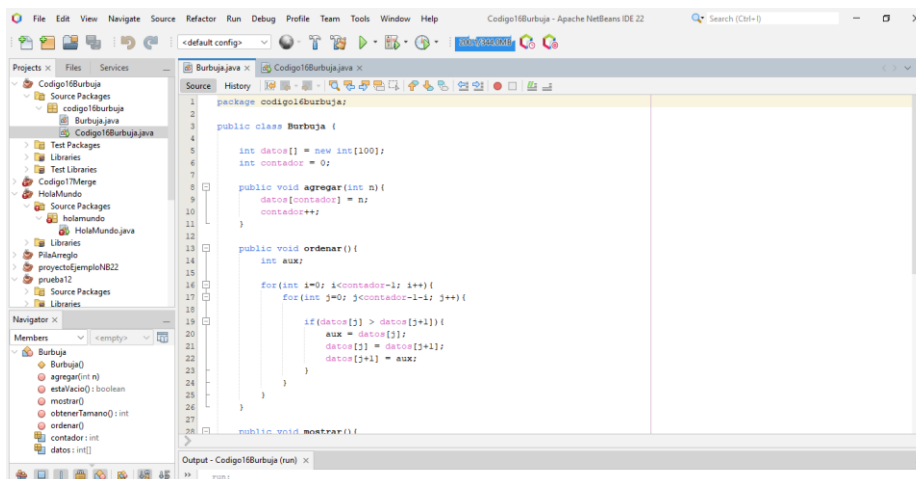
Manejo de una pila de manera dinámica por medio de las librerías o clases proporcionadas por el IDE.

I.16. Ordenamiento burbuja

El ordenamiento burbuja (Bubble Sort) es un algoritmo básico utilizado para organizar una lista de datos, como números o palabras, en orden ascendente o descendente. Su funcionamiento consiste en recorrer la lista varias veces, comparando pares de elementos consecutivos. Si los elementos están en el orden incorrecto, se intercambian de posición.



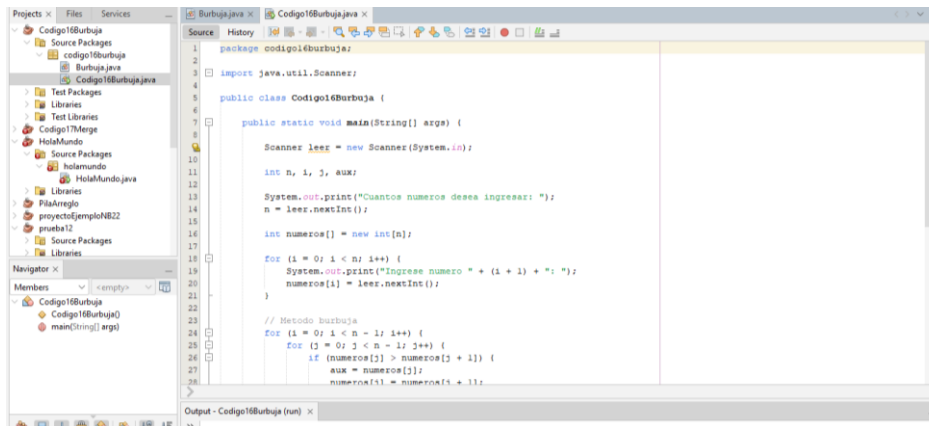
El programa compara cada elemento con el siguiente e intercambia posiciones cuando es necesario, repitiendo el proceso hasta que todos los valores quedan ordenados correctamente.



Cuenta con una variable llamada contador, la cual registra cuántos datos han sido ingresados.

El método agregar(int n) permite insertar números en el arreglo y aumentar el contador.

Posteriormente, el método ordenar() realiza comparaciones entre elementos consecutivos y, si están en desorden, los intercambia de posición usando una variable auxiliar. Este proceso se repite varias veces hasta acomodar todos los números de menor a mayor.



```
1 package codigo16burbuja;
2
3 import java.util.Scanner;
4
5 public class Codigo16Burbuja {
6
7     public static void main(String[] args) {
8
9         Scanner leer = new Scanner(System.in);
10
11         int n, i, j, aux;
12
13         System.out.print("Cuántos numeros desea ingresar: ");
14         n = leer.nextInt();
15
16         int numeros[] = new int[n];
17
18         for (i = 0; i < n; i++) {
19             System.out.print("Ingrese numero " + (i + 1) + ": ");
20             numeros[i] = leer.nextInt();
21         }
22
23         // Metodo burbuja
24         for (i = 0; i < n - 1; i++) {
25             for (j = 0; j < n - i - 1; j++) {
26                 if (numeros[j] > numeros[j + 1]) {
27                     aux = numeros[j];
28                     numeros[j] = numeros[j + 1];
29                     numeros[j + 1] = aux;
30                 }
31             }
32         }
33     }
34 }
```

Primero importa la clase Scanner, la cual permite capturar datos desde el teclado. Después, en el método principal main, solicita cuántos números desea ingresar el usuario y crea un arreglo con ese tamaño para almacenarlos.

Posteriormente, mediante un ciclo for, se piden uno por uno los números y se guardan en el arreglo. Una vez capturados, el programa aplica el método burbuja, comparando elementos consecutivos e intercambiándolos cuando están en desorden, utilizando una variable auxiliar.

Burbuja POO.

El método pedirDatos() solicita cuántos números se desean ingresar y posteriormente pide cada valor para almacenarlo en el arreglo arr[]. Después, el método burbuja() aplica el algoritmo de ordenamiento, comparando elementos consecutivos e intercambiándolos cuando el número de la izquierda es mayor que el de la derecha, usando una variable temporal llamada temp.

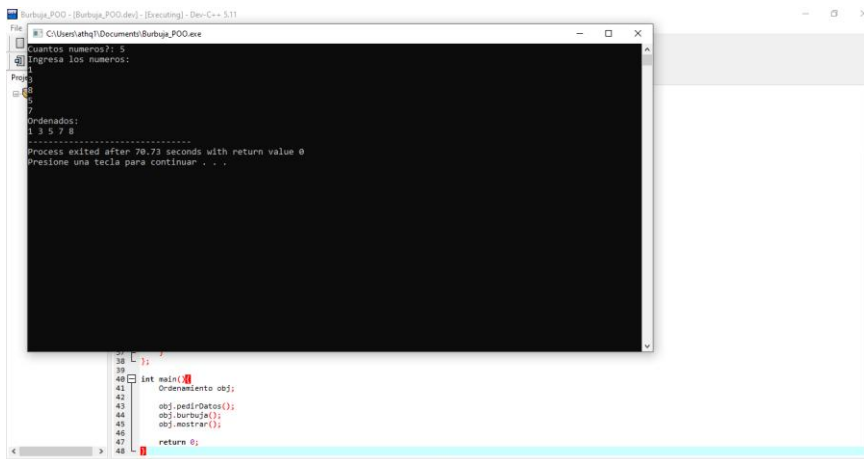


```
0 public:
1 void pedirDatos()
2 {
3     cout << "Cuántos numeros: ";
4     cin >> n;
5     cout << "Ingresar los numeros:\n";
6     for(int i=0; i<n; i++){
7         cin >> arr[i];
8     }
9 }
10
11 void burbuja()
12 {
13     for(int i=0; i<n-1; i++){
14         for(int j=i; j<n-1-i; j++){
15             if(arr[j] > arr[j+1]){
16                 int temp = arr[j];
17                 arr[j] = arr[j+1];
18                 arr[j+1] = temp;
19             }
20         }
21     }
22 }
23
24 void mostrar()
25 {
26     cout << "Ordenados:\n";
27     for(int i=0; i<n; i++){
28         cout << arr[i] << " ";
29     }
30 }
31
32 int main()
33 {
34     Ordenamiento obj;
35     obj.pedirDatos();
36     obj.burbuja();
37     obj.mostrar();
38     return 0;
39 }
```

Se abre una ventana de consola donde primero solicita al usuario la cantidad de números que desea ingresar. En la imagen se observa que se escribió el número 7, por lo que el programa pidió capturar siete valores.

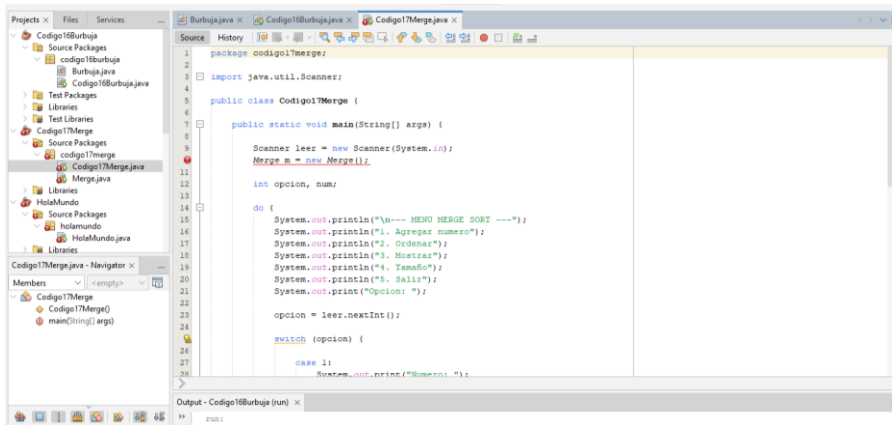
Después, el usuario ingresó los números 2, 9, 6, 1, 5, 4 y 3, los cuales fueron almacenados en el arreglo. Una vez capturados todos los datos, el programa ejecutó automáticamente el método de ordenamiento Burbuja, comparando los números entre sí e intercambiándolos hasta acomodarlos correctamente de menor a mayor.

Finalmente, el programa mostró en pantalla el resultado ordenado: 1 2 3 4 5 6 9



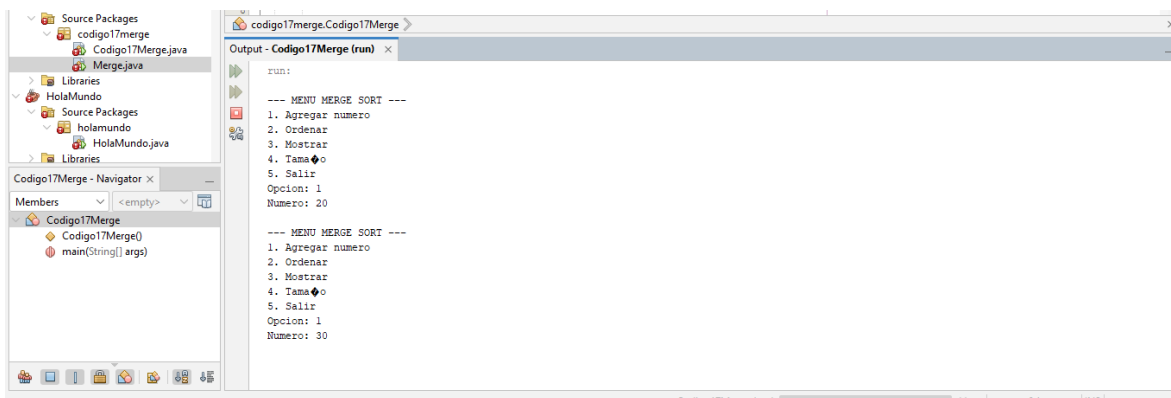
```
File Edit View Help
Burbuja_POO - [Burbuja_POO.dev] - [Executing] - Dev-C++ 5.11
C:\Users\ahq\Documents\Burbuja_POO.exe
Cuántos numeros?: 7
Ingresar los numeros:
2
9
6
1
5
4
3
Ordenados:
1 2 3 4 5 6 9
Process exited after 70.73 seconds with return value 0
Presione una tecla para continuar . . .
```


17.Ordenamiento Merge Sort

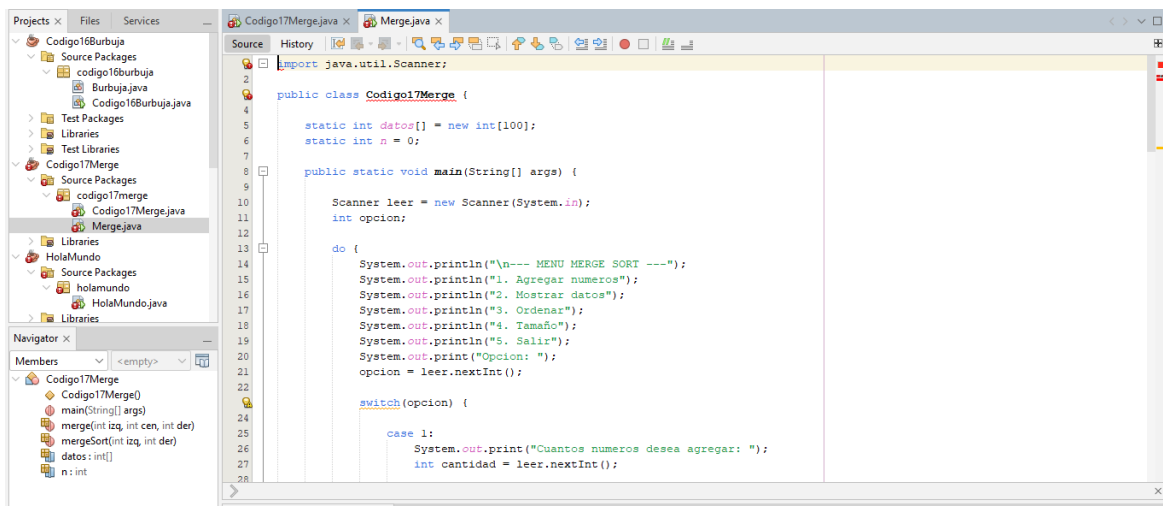


Como corrección de este código, se realizó nuevamente en el lenguaje de programación Java, utilizando una estructura más ordenada y funcional para ejecutar correctamente el método de ordenamiento burbuja. Se empleó la clase Scanner para la entrada de datos desde el teclado y se mejoró la lógica del programa para evitar errores en la captura y ordenamiento de los números.

Se utilizaron las siguientes variables: n para almacenar la cantidad de números a ingresar, i y j como contadores de los ciclos repetitivos, aux como variable auxiliar para intercambiar valores durante el ordenamiento, y el arreglo numeros[] para guardar los datos introducidos por el usuario.



Las opciones del menú son: 1. Agregar número, que permite ingresar valores al arreglo; 2. Ordenar, que aplica el método Merge Sort para acomodar los datos de menor a mayor; 3. Mostrar, que enseña los números almacenados; 4. Tamaño, que indica cuántos elementos se han guardado; y 5. Salir, que finaliza el programa.

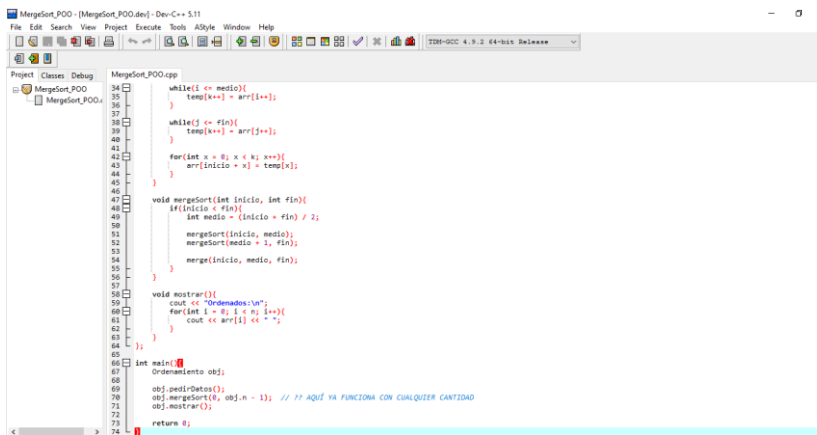


Se declararon las variables `datos[]`, que es un arreglo con capacidad para 100 números, `n`, que lleva el conteo de elementos guardados, y `opcion`, que almacena la selección elegida en el menú. Mediante un ciclo `do-while`, el programa muestra continuamente las opciones hasta que el usuario decida salir.

Merege sort POO.

Primero, el método `pedirDatos()` solicita cuántos números se desean ingresar y guarda cada valor dentro del arreglo `arr[]`. Después, el método `mergeSort(int inicio, int fin)` se encarga de dividir el arreglo en partes pequeñas de manera recursiva, hasta separar los elementos individualmente.

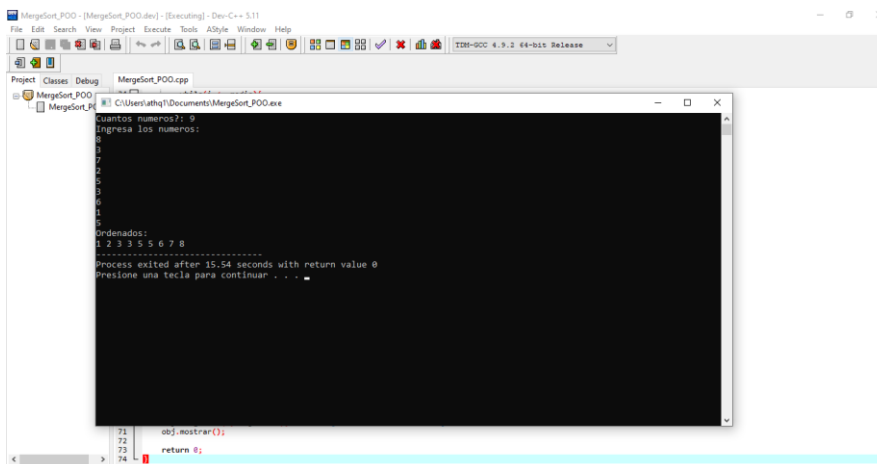
Posteriormente, la función `merge(inicio, medio, fin)` une nuevamente las partes ya divididas, comparando sus valores y acomodándolos de menor a mayor en un arreglo temporal llamado `temp[]`. Este proceso continúa hasta reconstruir todo el arreglo completamente ordenado.



```
34 while(i <= medio){
35     temp[k++] = arr[i++];
36 }
37 while(j <= fin){
38     temp[k++] = arr[j++];
39 }
40 for(int x = 0; x < k; x++){
41     arr[inicio + x] = temp[x];
42 }
43 }
44
45 void mergeSort(int inicio, int fin){
46     if(inicio < fin){
47         int medio = (inicio + fin) / 2;
48         mergeSort(inicio, medio);
49         mergeSort(medio + 1, fin);
50         merge(inicio, medio, fin);
51     }
52 }
53
54 void mostrar(){
55     cout << "Ordenados:\n";
56     for(int i = 0; i < n; i++){
57         cout << arr[i] << " ";
58     }
59 }
60
61 int main(){
62     Ordenamiento obj;
63     obj.pedirDatos();
64     obj.mergeSort(0, obj.n - 1); // ?? AQUÍ YA FUNCIONA CON CUALQUIER CANTIDAD
65     obj.mostrar();
66     return 0;
67 }
```

Se escribió 9, por lo que el sistema pidió capturar nueve valores.

Después, se ingresaron los números 8, 3, 7, 2, 5, 3, 6, 1 y 5, los cuales fueron almacenados en el arreglo. Una vez completada la captura, el programa ejecutó automáticamente el método Merge Sort, que divide el arreglo en partes pequeñas, ordena cada sección y después las une nuevamente en orden ascendente.



```
71 obj.mostrar();
72
73 return 0;
74
```

18. Ordenamiento Quick Sort (JAVA)

El objetivo de este programa es implementar el algoritmo de ordenamiento *Quick Sort* (Ordenamiento Rápido) para organizar una colección de un "Nuevo Tipo de Dato" mediante el paradigma Orientado a Objetos (POO).

uno con los elementos menores al pivote y otro con los mayores, aplicando este proceso de forma recursiva. Para este ejercicio, el programa ordena una lista de objetos tipo *Persona* basándose estrictamente en su atributo *edad*, de menor a mayor.

OrdenadorQuick.java

```
1 public class OrdenadorQuick extends ADTOrdenadorNTD { 1 usage
2     @Override 1 usage
3     public void ordenarPorEdad(Persona[] array, int tamActual) {
4         if (array == null || tamActual <= 1) {
5             return;
6         }
7         quickSortRecursivo(array, inicio: 0, fin: tamActual - 1);
8     }
9     private void quickSortRecursivo(Persona[] array, int inicio, int fin) {
10        if (inicio < fin) {
11            int indicePivote = particion(array, inicio, fin);
12            quickSortRecursivo(array, inicio, fin: indicePivote - 1);
13            quickSortRecursivo(array, inicio: indicePivote + 1, fin);
14        }
15    }
16    @ private int particion(Persona[] array, int inicio, int fin) { 1 usage
17        int edadPivote = array[fin].edad;
18        int i = inicio - 1;
19
20        for (int j = inicio; j < fin; j++) {
21            if (array[j].edad <= edadPivote) {
22                i++;
23                Persona temp = array[i];
24                array[i] = array[j];
25                array[j] = temp;
26            }
27        }
28        Persona temp2 = array[i + 1];
29        array[i + 1] = array[fin];
30        array[fin] = temp2;
31
32        return i + 1;
33    }
```

Persona.java

```
1 public class Persona { 9 usages
2     public String nombre; 2 usages
3     public int edad; 4 usages
4
5     public Persona(String nombre, int edad) { 1 usage
6         this.nombre = nombre;
7         this.edad = edad;
8     }
9
10    @Override
11    public String toString() {
12        return "Persona {" + "nombre=" + nombre + '\n' +
13            ", edad=" + edad + '}' + '\n';
14    }
15 }
```

ADTOrdenadorNTD.java

```
1 public abstract class ADTOrdenadorNTD { 2 usages 1 inheritor
2     public abstract void ordenarPorEdad(Persona[] array, int tamActual);
3 }
```

La gran ventaja matemática y técnica de esta implementación es que se realiza *In-Place* (en el mismo lugar). A diferencia de otros enfoques lógicos que requieren crear arreglos nuevos para guardar los datos ordenados (lo cual duplicaría el consumo de memoria RAM), nuestra implementación de Quick Sort reacomoda a los objetos dentro del mismo arreglo original. El método particion selecciona siempre a la última persona del sub-arreglo como pivote. Posteriormente, mediante un ciclo for y una variable temporal de tipo Persona, empuja a todos los objetos con menor edad hacia la izquierda del pivote, garantizando que este último quede en su posición final definitiva. Finalmente, la recursividad se encarga de repetir el proceso con las mitades restantes.

Hubo errores, uno de ellos fue el de recursividad en los límites del arreglo:

El problema: Inicialmente, al hacer las llamadas recursivas para ordenar la mitad izquierda y derecha, se utilizaron los límites inicio a fin - 1 y fin + 1 a fin. Esto provocaba que el arreglo no se dividiera correctamente.

La solución: Se comprendió que el punto de corte no lo define la variable fin, sino la variable indicePivote (la posición exacta donde quedó acomodado nuestro pivote actual). Se corrigió la lógica para que la mitad izquierda termine en indicePivote - 1 y la mitad derecha empiece en indicePivote + 1.

Otro de los errores fue de POO en el intercambio de variables:

El problema: Al momento de hacer el intercambio de posiciones, el código original intentaba intercambiar solo los atributos enteros `int temp = array[i].edad`. Este no fue error como tal en la lógica de como se hace, si se logro una parte del objetivo que fue intercambiar datos, sin embargo, solo se hacia con la edad, el nombre aun pertenecía intacto y se estropea lo que se quiere llegar a lograr, debido que si tenia datos como Daniel 19, Pepe 20, solo conseguíamos tener Daniel 20, Pepe 19

La solución: Se corrigió el tipo de dato de la variable temporal, cambiando de un tipo básico (`int temp`) al Nuevo Tipo de Dato (`Persona temp = array[i]`) de la línea 16 de la clase OrdenadorQuick . Esto garantizó que, al hacer el intercambio en el arreglo, se moviera el objeto Persona completo (con su nombre y edad intactos) a su nueva posición en la memoria.

```
Persona registrada correctamente
Ingresa el nombre (sin espacios): pepe
Ingresa la edad: 12
Persona registrada correctamente

<=== MENU QUICK SORT ===>
1. Agregar persona. (aquí se ponen desordenadas a proposito
con finalidad de tener el cambio con QuickSort
y verlas posteriormente ordenadas con la opcion 3 despues con la opcion 2 para mostrar contenido)
2. Mostrar contenido de la lista
3. aplicar ordenamiento por edad con QuickSort
4. Salir
Elige una opcion: 3

Procesando el algoritmo Quick Sort...
Lista ordenada con exito, usa la opcion 2 para ver el resultado.

<=== MENU QUICK SORT ===>
1. Agregar persona. (aquí se ponen desordenadas a proposito
con finalidad de tener el cambio con QuickSort
y verlas posteriormente ordenadas con la opcion 3 despues con la opcion 2 para mostrar contenido)
2. Mostrar contenido de la lista
3. aplicar ordenamiento por edad con QuickSort
4. Salir
Elige una opcion: 2

--- LISTA ACTUAL ---
Persona {nombre='pepe', edad=12}
Persona {nombre='daniel', edad=24}
```

II. Pila

¿Qué es una pila?

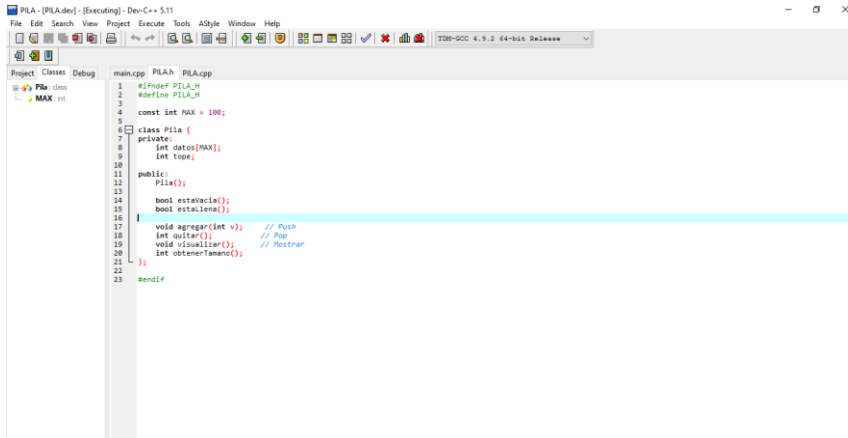
Una pila es una estructura de datos utilizada en programación para almacenar elementos siguiendo el principio LIFO (Last In, First Out), que significa el último en entrar es el primero en salir. Su funcionamiento es similar a una pila de platos: el último plato que se coloca arriba es el primero que se retira.

Las operaciones principales de una pila son push, que sirve para agregar un elemento en la parte superior, y pop, que elimina el elemento que se encuentra arriba. También existe la operación peek o top, que permite consultar el elemento superior sin eliminarlo.

En este código se realizaron varias correcciones durante su desarrollo para lograr un funcionamiento adecuado, principalmente en la interfaz y en la conexión con la lógica del programa. Fue necesario ajustar errores relacionados con botones, ventanas, entrada y salida de datos, así como detalles en la presentación visual para que la interacción con el usuario fuera más clara y ordenada.

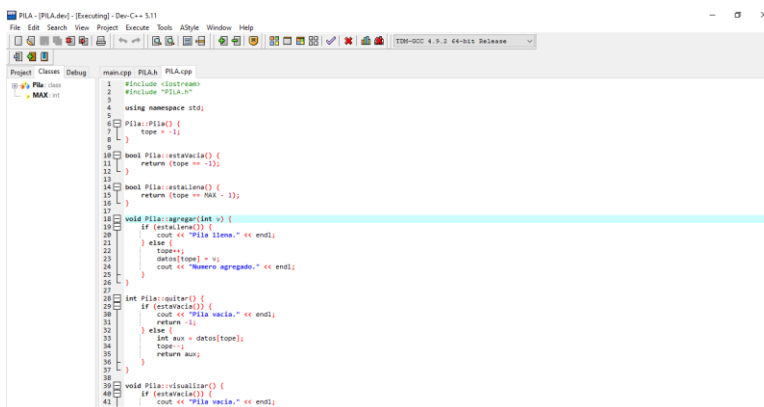
También se corrigieron problemas en la estructura del código, declaración de variables, métodos y llamadas entre clases, permitiendo que las funciones de la pila trabajaran correctamente dentro de la interfaz gráfica. Además, se mejoró la organización del programa para evitar fallos al ejecutar operaciones como insertar, eliminar y mostrar datos.

1. Pila arreglo



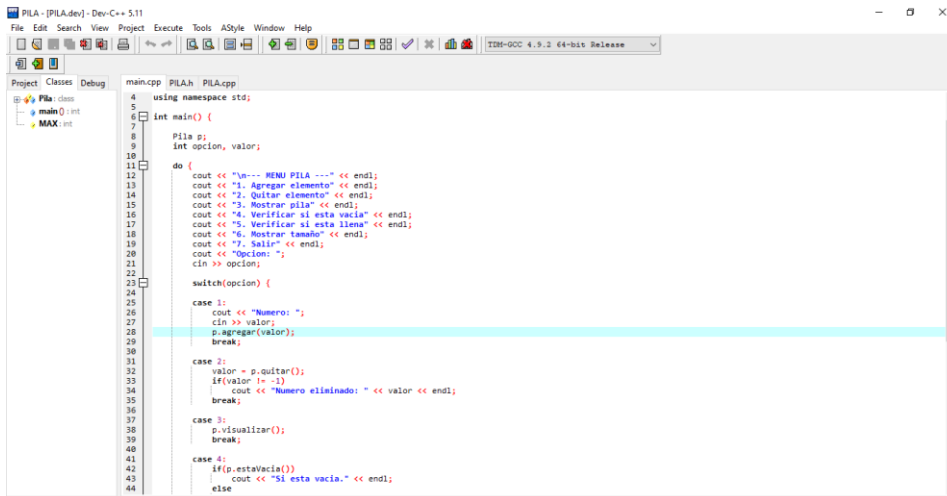
El archivo de encabezado **PILA.h**, donde se define la estructura principal de una pila utilizando Programación Orientada a Objetos. En este archivo se declara una constante llamada MAX = 100, que indica la capacidad máxima de elementos que puede almacenar la pila.

También se crea la clase Pila, la cual contiene en la parte privada un arreglo llamado datos[MAX], encargado de guardar los valores, y la variable tope, que sirve para señalar la posición del último elemento ingresado.



El funcionamiento de esta parte primero, el constructor Pila::Pila() inicializa la variable tope en **-1**, lo que indica que la pila comienza vacía. Después, el método estaVacia() verifica si no hay elementos almacenados, mientras que estaLlena() comprueba si ya se alcanzó el límite máximo definido por MAX.

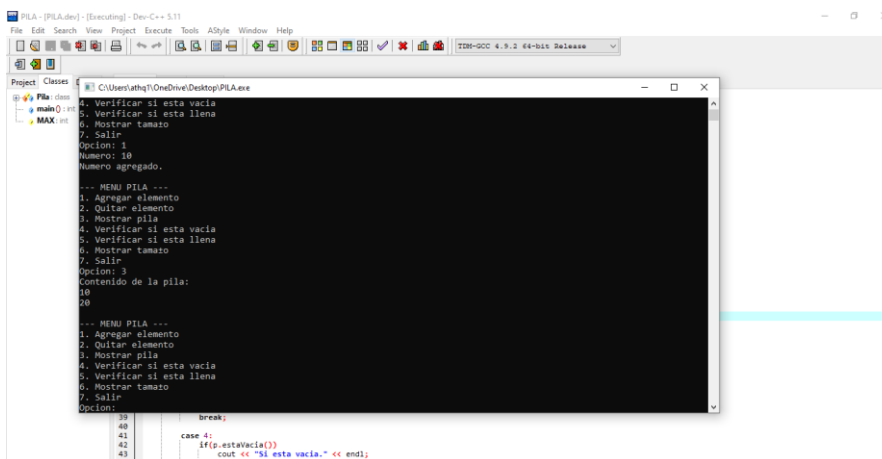
El método `agregar(int v)` realiza la operación **Push**, es decir, insertar un nuevo elemento en la pila. Si la pila está llena, muestra el mensaje “Pila llena”; de lo contrario, incrementa la variable `tope`, guarda el valor en el arreglo `datos[]` y confirma que el número fue agregado.



```
4 using namespace std;
5
6 int main() {
7     Pila p;
8     int opcion, valor;
9
10    do {
11        cout << "Menu PILA ---" << endl;
12        cout << "1. Agregar elemento" << endl;
13        cout << "2. Quitar elemento" << endl;
14        cout << "3. Mostrar pila" << endl;
15        cout << "4. Verificar si esta vacia" << endl;
16        cout << "5. Verificar si esta llena" << endl;
17        cout << "6. Mostrar tamaño" << endl;
18        cout << "7. Salir" << endl;
19        cout << "Opcion: ";
20        cin >> opcion;
21
22        switch(opcion) {
23
24            case 1:
25                cout << "Numero: ";
26                cin >> valor;
27                p.agregar(valor);
28                break;
29
30            case 2:
31                valor = p.quitar();
32                if(valor != -1)
33                    cout << "Numero eliminado: " << valor << endl;
34                break;
35
36            case 3:
37                p.visualizar();
38                break;
39
40            case 4:
41                if(p.estaVacia())
42                    cout << "Si esta vacia." << endl;
43                else
44                    break;
```

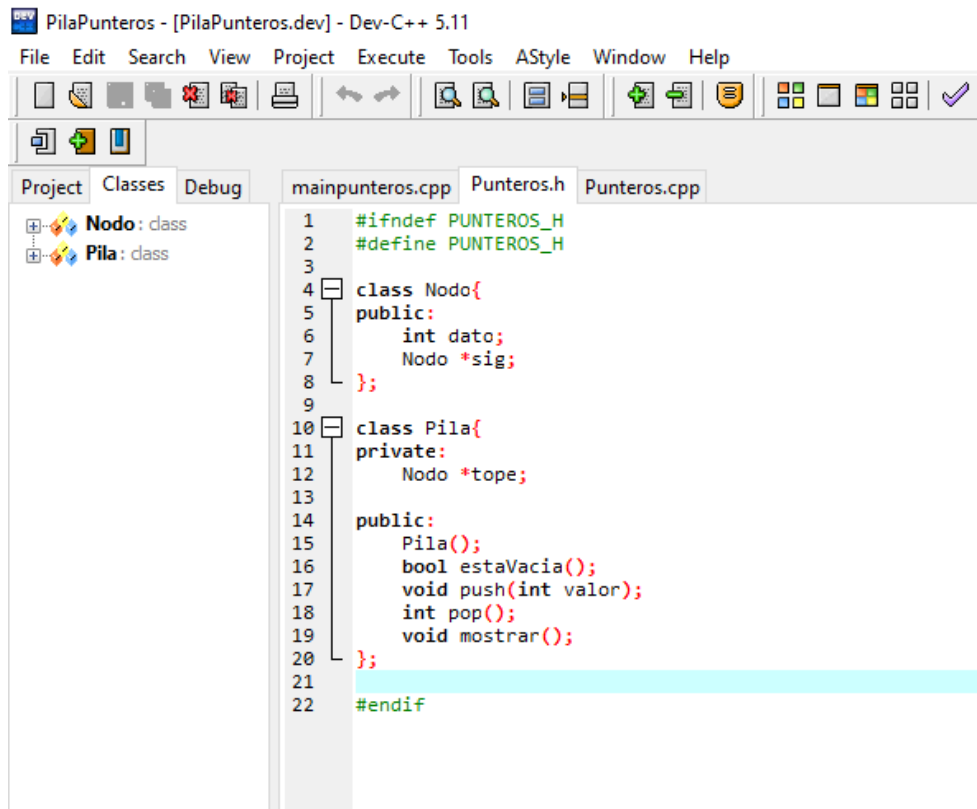
Se declaran las variables `opcion` y `valor`, utilizadas para controlar el menú y capturar números ingresados por el usuario. Después, mediante un ciclo `do-while`, el programa muestra continuamente un menú interactivo con varias opciones hasta que el usuario decida salir.

Las opciones del menú son: 1. Agregar elemento, que inserta un número en la pila mediante `p.agregar(valor)`; 2. Quitar elemento, que elimina el último dato ingresado usando `p.quitar()`; 3. Mostrar pila, que enseña todos los elementos almacenados; 4. Verificar si está vacía, que indica si no contiene datos; 5. Verificar si está llena, que revisa si alcanzó su capacidad máxima; 6. Mostrar tamaño, que indica cuántos elementos tiene; y 7. Salir, que termina el programa.



```
1. Verificar si esta vacia
2. Verificar si esta llena
3. Mostrar tamaño
7. Salir
opcion: 1
Numero: 10
Numero agregado.
--- MENU PILA ---
1. Agregar elemento
2. Quitar elemento
3. Mostrar pila
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tamaño
7. Salir
opcion: 3
Contenido de la pila:
10
--- MENU PILA ---
1. Agregar elemento
2. Quitar elemento
3. Mostrar pila
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tamaño
7. Salir
opcion: 4
Si esta vacia.
```

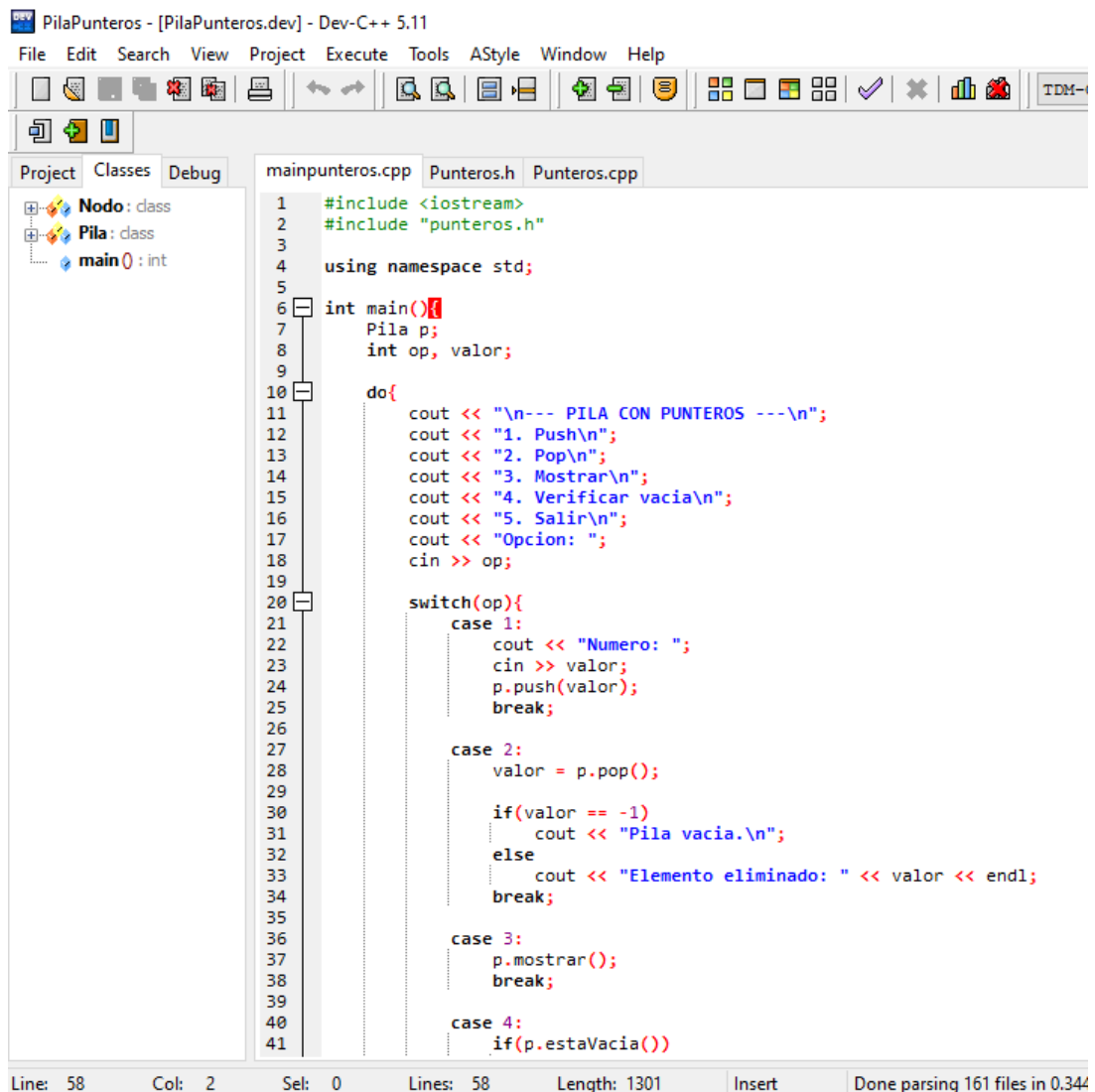

2. Pila punteros



```
PilaPunteros - [PilaPunteros.dev] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
[Icons]
Project Classes Debug
+ Nodo: class
+ Pila: class
mainpunteros.cpp Punteros.h Punteros.cpp
1  #ifndef PUNTEROS_H
2  #define PUNTEROS_H
3
4  class Nodo{
5  public:
6      int dato;
7      Nodo *sig;
8  };
9
10 class Pila{
11 private:
12     Nodo *tope;
13
14 public:
15     Pila();
16     bool estaVacia();
17     void push(int valor);
18     int pop();
19     void mostrar();
20 };
21
22 #endif
```

Este código corresponde al archivo Punteros.h, donde se definen las clases necesarias para crear una pila usando punteros, en lugar de arreglos fijos. Esto permite que la memoria se utilice dinámicamente conforme se agregan datos.

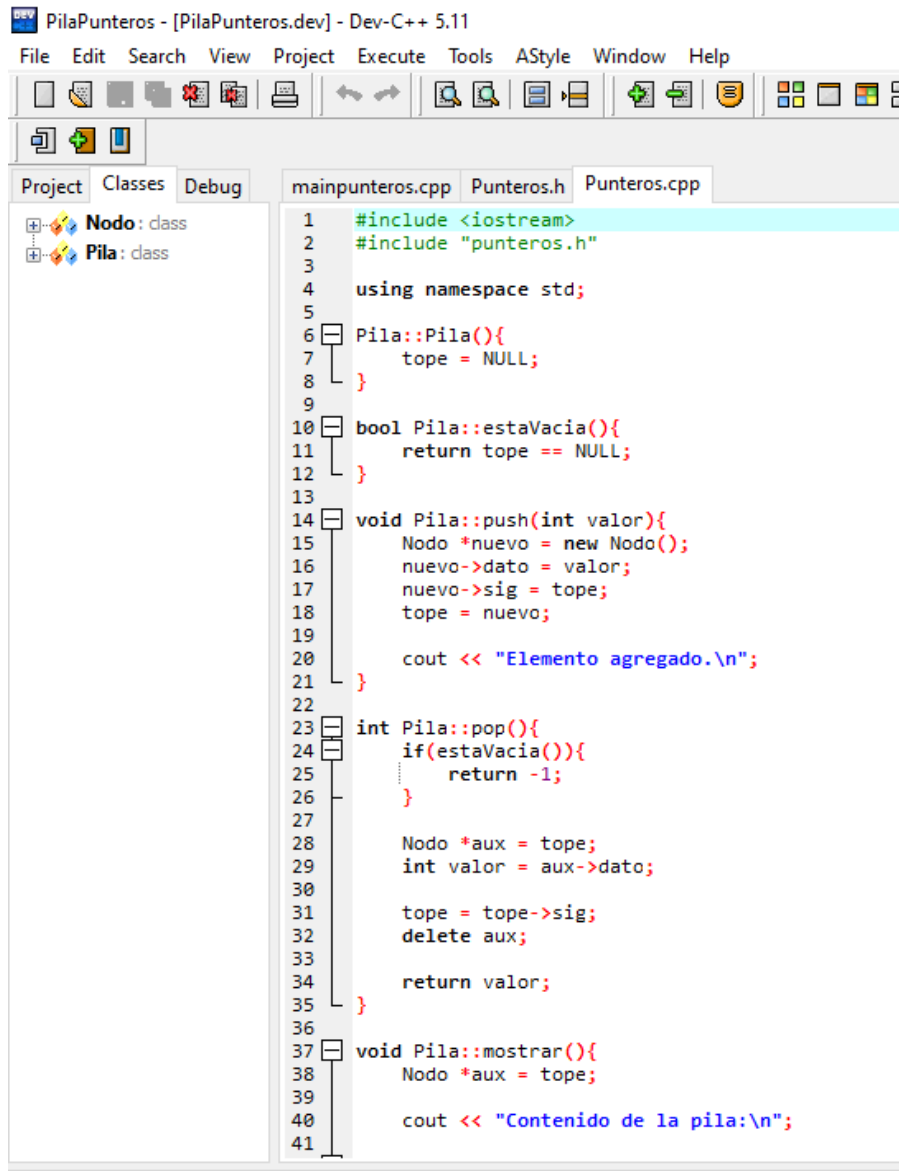
Primero se declara la clase `Nodo`, la cual representa cada elemento de la pila. Dentro de ella se encuentra la variable `dato`, que guarda el valor almacenado, y el puntero `sig`, que apunta al siguiente nodo de la estructura.



```
1  #include <iostream>
2  #include "punteros.h"
3
4  using namespace std;
5
6  int main(){
7      Pila p;
8      int op, valor;
9
10     do{
11         cout << "\n--- PILA CON PUNTEROS ---\n";
12         cout << "1. Push\n";
13         cout << "2. Pop\n";
14         cout << "3. Mostrar\n";
15         cout << "4. Verificar vacia\n";
16         cout << "5. Salir\n";
17         cout << "Opcion: ";
18         cin >> op;
19
20         switch(op){
21             case 1:
22                 cout << "Numero: ";
23                 cin >> valor;
24                 p.push(valor);
25                 break;
26
27             case 2:
28                 valor = p.pop();
29
30                 if(valor == -1)
31                     cout << "Pila vacia.\n";
32                 else
33                     cout << "Elemento eliminado: " << valor << endl;
34                 break;
35
36             case 3:
37                 p.mostrar();
38                 break;
39
40             case 4:
41                 if(p.estaVacia())
```

Primero se crea un objeto llamado p de la clase Pila, además de las variables op y valor, que sirven para guardar la opción elegida por el usuario y los números que se desean ingresar.

Después, mediante un ciclo do-while, el programa muestra constantemente un menú con varias opciones hasta que el usuario decida salir. Las opciones disponibles son: 1. Push, que inserta un nuevo elemento en la pila con p.push(valor); 2. Pop, que elimina el elemento ubicado en la cima mediante p.pop(); 3. Mostrar, que enseña los datos almacenados; 4. Verificar vacía, que indica si la pila no contiene elementos; y 5. Salir, que finaliza la ejecución.



```
1 #include <iostream>
2 #include "punteros.h"
3
4 using namespace std;
5
6 Pila::Pila(){
7     tope = NULL;
8 }
9
10 bool Pila::estaVacia(){
11     return tope == NULL;
12 }
13
14 void Pila::push(int valor){
15     Nodo *nuevo = new Nodo();
16     nuevo->dato = valor;
17     nuevo->sig = tope;
18     tope = nuevo;
19
20     cout << "Elemento agregado.\n";
21 }
22
23 int Pila::pop(){
24     if(estaVacia()){
25         return -1;
26     }
27
28     Nodo *aux = tope;
29     int valor = aux->dato;
30
31     tope = tope->sig;
32     delete aux;
33
34     return valor;
35 }
36
37 void Pila::mostrar(){
38     Nodo *aux = tope;
39
40     cout << "Contenido de la pila:\n";
41 }
```

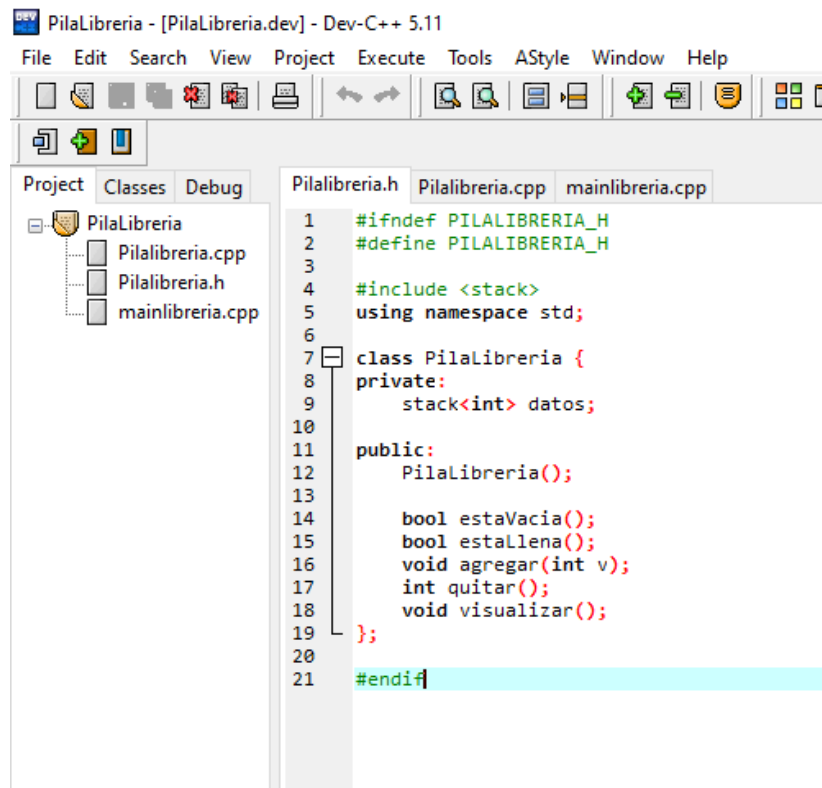
El constructor `Pila::Pila()` inicializa el puntero `tope` en `NULL`, indicando que la pila comienza vacía. Después, el método `estaVacia()` verifica si no existe ningún nodo almacenado.

El método `push(int valor)` realiza la operación de insertar un nuevo elemento. Para ello crea dinámicamente un nodo con `new`, guarda el valor en `dato`, enlaza el nuevo nodo con el anterior mediante `sig`, y actualiza `tope` para que ahora apunte al nuevo elemento agregado.

3. Pila Librería

Se corrigieron varios errores durante el desarrollo del código, principalmente en la declaración de la clase y en la organización del archivo de encabezado. Se implementó la directiva de protección `#ifndef` y `#define` para evitar duplicidad al compilar el programa. Posteriormente, se agregó la librería `<stack>` para utilizar la pila proporcionada por C++ y facilitar el manejo de datos.

En este código se creó la clase `PilaLibreria`, la cual contiene de manera privada una pila de tipo entero llamada `datos`, donde se almacenan los elementos ingresados por el usuario. Después, en la parte pública, se declararon los métodos principales para el funcionamiento de la estructura: constructor, verificación de pila vacía, comprobación de pila llena, agregar elementos, quitar elementos y visualizar el contenido.

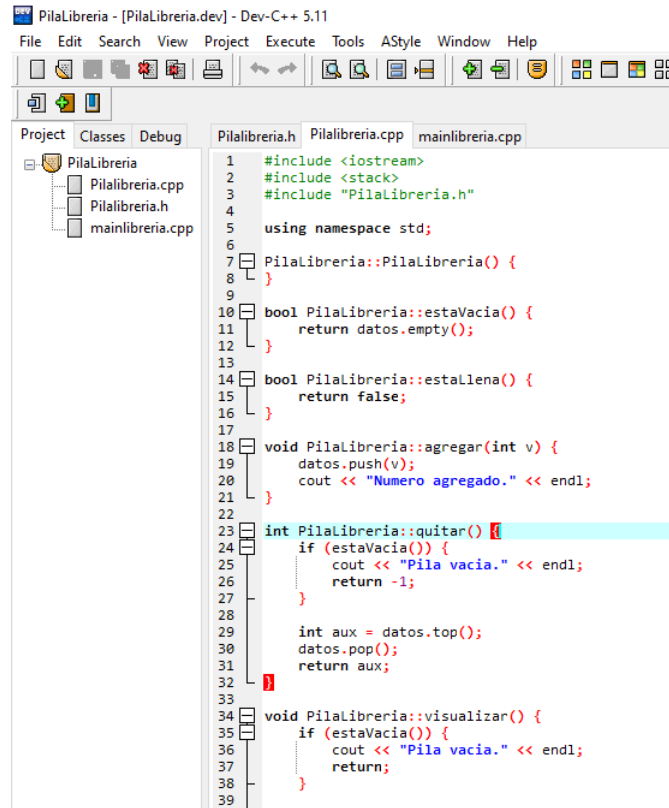


```
PilaLibreria - [PilaLibreria.dev] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help

Project Classes Debug
PilaLibreria
├── PilaLibreria.cpp
├── PilaLibreria.h
└── mainlibreria.cpp

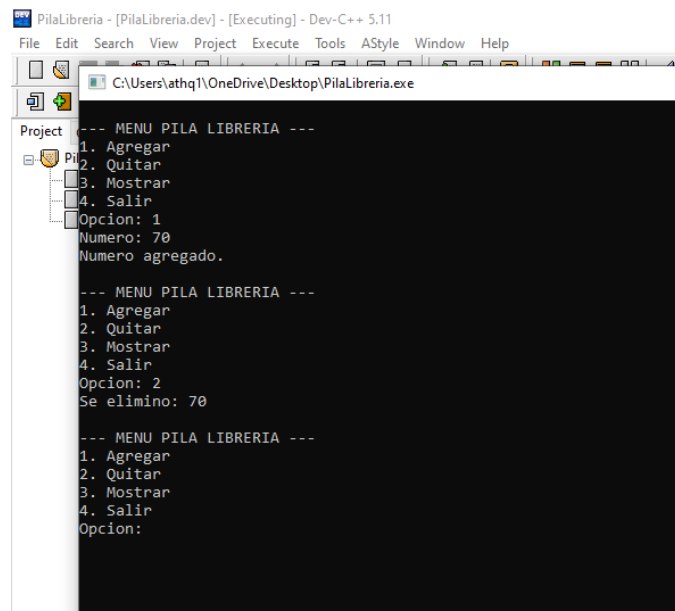
PilaLibreria.h
1  #ifndef PILALIBRERIA_H
2  #define PILALIBRERIA_H
3
4  #include <stack>
5  using namespace std;
6
7  class PilaLibreria {
8  private:
9      stack<int> datos;
10
11 public:
12     PilaLibreria();
13
14     bool estaVacia();
15     bool estaLlena();
16     void agregar(int v);
17     int quitar();
18     void visualizar();
19 };
20
21 #endif
```

En primer lugar, se implementó el constructor `PilaLibreria()`, encargado de inicializar el objeto al momento de ser creado. Que nos sirve para la implementación de la variable `estaVacia()`, a cual utiliza el método `empty()` para verificar si la pila contiene elementos o no. También se agregó la función `estaLlena()`, que retorna `false`, ya que la pila basada en librería estándar no tiene un límite fijo de tamaño como en los arreglos.



```
1 #include <iostream>
2 #include <stack>
3 #include "Pilalibreria.h"
4
5 using namespace std;
6
7 Pilalibreria::Pilalibreria() {
8 }
9
10 bool Pilalibreria::estaVacia() {
11     return datos.empty();
12 }
13
14 bool Pilalibreria::estaLlena() {
15     return false;
16 }
17
18 void Pilalibreria::agregar(int v) {
19     datos.push(v);
20     cout << "Numero agregado." << endl;
21 }
22
23 int Pilalibreria::quitar() {
24     if (estaVacia()) {
25         cout << "Pila vacia." << endl;
26         return -1;
27     }
28
29     int aux = datos.top();
30     datos.pop();
31     return aux;
32 }
33
34 void Pilalibreria::visualizar() {
35     if (estaVacia()) {
36         cout << "Pila vacia." << endl;
37         return;
38     }
39 }
```

La función de este código se crea un objeto llamado p, el cual representa la pila que almacenará los datos ingresados por el usuario. También se declaran variables auxiliares como op, utilizada para guardar la opción del menú, y num, que almacena los números capturados, mediante una estructura repetitiva do while, se muestra continuamente un menú con cuatro opciones: agregar, quitar, mostrar y salir. Esto permite que el usuario pueda interactuar varias veces con la pila sin necesidad de reiniciar el programa.



```
--- MENU PILA LIBRERIA ---
1. Agregar
2. Quitar
3. Mostrar
4. Salir
Opcion: 1
Numero: 70
Numero agregado.

--- MENU PILA LIBRERIA ---
1. Agregar
2. Quitar
3. Mostrar
4. Salir
Opcion: 2
Se elimino: 70

--- MENU PILA LIBRERIA ---
1. Agregar
2. Quitar
3. Mostrar
4. Salir
Opcion:
```

III. Cola

4. Array

Objetivo

El objetivo de esta implementación es simular el comportamiento de una estructura de datos tipo **cola (FIFO: First In, First Out)** utilizando un **arreglo estático de tamaño fijo**, permitiendo realizar operaciones básicas como inserción, eliminación, consulta y recorrido de elementos.

Análisis

Una cola es una estructura lineal donde el primer elemento en entrar es el primero en salir.

En esta implementación, la cola se gestiona mediante un arreglo, por lo que es necesario controlar manualmente las posiciones de inserción y eliminación.

Para ello, se utilizan las siguientes variables:

- **frente:** indica la posición del primer elemento
- **fin:** indica la posición del último elemento
- **cantidad:** número de elementos actuales en la cola
- **capacidad:** tamaño máximo del arreglo

El uso de estas variables permite simular el comportamiento de una cola sin necesidad de estructuras dinámicas.

A. Dato Básico (DB)

En esta fase, la cola almacena números enteros (int).

- **Archivos analizados:** ColaArregloDB_Abstrakta.java y ColaArregloDB_Concreta.java.
- **Estructura:** Se utilizó una **Clase Abstracta** para definir los métodos obligatorios (agregar, quitar, mostrar, estaVacía, estaLlena). La clase concreta hereda estos atributos y aplica la lógica matemática para mover el puntero $fin = (fin + 1) \% capacidad$.

B. Nuevo Tipo de Dato (ND)

Aquí elevamos la complejidad aplicando el paradigma **Orientado a Objetos (POO)**. La cola ahora gestiona objetos de la clase Producto.

- **Archivos analizados:** Producto.java, ColaArregloND_Abstrakta.java y ColaArregloND_Concreta.java.
- **Lógica:** Cada elemento de la cola es una instancia que contiene nombre y precio. El método `quitar()` no solo reduce el contador cantidad, sino que imprime el objeto eliminado gracias al método `toString()` de la clase Producto.

✓ Inserción (agregar)

- Se verifica si la cola está llena
- Se incrementa la variable `fin`
- Se inserta el elemento en el arreglo
- Se incrementa cantidad

✓ Eliminación (quitar)

- Se valida si la cola está vacía
- Se elimina el elemento en la posición frente
- Se recorren los elementos hacia la izquierda
- Se decrementa cantidad

✓ Mostrar

- Se recorre el arreglo desde frente hasta cantidad
- Se imprimen los elementos en orden

✓ Validaciones

- **estaVacía()**: retorna verdadero si `cantidad == 0`
- **estaLlena()**: retorna verdadero si `cantidad == capacidad`

4.5 Resolución de Problemas

Durante la implementación se resolvieron los siguientes aspectos:

✓ Control de desbordamiento (Overflow)

Se evita insertar elementos cuando la cola está llena mediante la validación:

`cantidad == capacidad`

✓ Control de subdesbordamiento (Underflow)

Se evita eliminar elementos cuando la cola está vacía:

`cantidad == 0`

✓ Organización de datos

Se utiliza un recorrido (shift) para mantener el orden después de eliminar un elemento.

ArregloNuevoDato

Main

```
1 package Arreglos.ArregloNuevoDato;
2
3 import java.util.Scanner;
4
5 public class main {
6     public static void main(String[] args) {
7
8         Scanner sc= new Scanner(System.in);
9
10        System.out.print("Ingresa la capacidad: ");
11
12        int cap = sc.nextInt();
13        sc.nextLine();
14
15        ColaArregloND_Concreta cola = new ColaArregloND_Concreta(cap);
16
17        int op;
18
19        do {
20            System.out.println("\n--- MENU ---");
21            System.out.println("1. Agregar producto");
22            System.out.println("2. Quitar producto");
23            System.out.println("3. Mostrar");
24            System.out.println("4. ¿Está vacía?");
25            System.out.println("5. ¿Está llena?");
26            System.out.println("6. Tamaño");
27            System.out.println("7. Salir");
28            System.out.print("Opción: ");
29            op = sc.nextInt();
30            sc.nextLine();
31
32            switch (op) {
```

ColaArregloND_abstracta

```
1 package Arreglos.ArregloNuevoData;
2
3 public abstract class ColaArregloND_Abstracta { 1 usage 1 inheritor
4     protected Producto[] datos; 4 usages
5     protected int frente; 5 usages
6     protected int fin; 4 usages
7     protected int capacidad;
8     protected int cantidad; 7 usages
9
10    public ColaArregloND_Abstracta(int capacidad) { 1 usage
11        this.capacidad = capacidad;
12        this.datos = new Producto[capacidad];
13        this.frente = 0;
14        this.fin = -1;
15        this.cantidad = 0;
16    }
17
18    public abstract void agregar(Producto p); 1 implementation
19    public abstract void quitar(); 1 implementation
20    public abstract void mostrar(); 1 implementation
21    public abstract boolean estaVacia(); 1 implementation
22    public abstract boolean estaLlena(); 1 implementation
23    public abstract int tamanio(); 1 usage 1 implementation
24
25 }
```

ColaArregloND_concreta

```
ArregloNuevoData\main.java  Producto.java  ColaArregloND_Concreta.java  ColaArregloND_Abstracta.java
1 package Arreglos.ArregloNuevoData;
2
3 public class ColaArregloND_Concreta extends ColaArregloND_Abstracta { 2 usages
4     public ColaArregloND_Concreta(int capacidad) { super(capacidad); }
5
6
7
8     @Override
9     public void agregar(Producto p) {
10         if (estaLlena()) {
11             System.out.println("La cola está llena.");
12         } else {
13             fin = (fin + 1) % capacidad;
14             datos[fin] = p;
15             cantidad++;
16         }
17     }
18
19     @Override
20     public void quitar() {
21         if (estaVacia()) {
22             System.out.println("La cola está vacia.");
23         } else {
24             System.out.println("Eliminado: " + datos[frente]);
25             frente = (frente + 1) % capacidad;
26             cantidad--;
27         }
28     }
29
30     @Override
31     public void mostrar() {
32         if (estaVacia()) {
33             System.out.println("Vacia");
34         } else {
35             int i = frente;
36             for (int c = 0; c < cantidad; c++) {
```

Producto



```
1 package Arreglos.ArregloNuevoDato;
2
3 public class Producto { 5 usages
4     private String nombre; 2 usages
5     private double precio; 2 usages
6
7     public Producto(String nombre, double precio) { 1 usage
8         this.nombre = nombre;
9         this.precio = precio;
10    }
11
12    public String toString() { return nombre + " - $" + precio; }
13
14 }
```

Errores Presentados

✗ Error 1: Desbordamiento del arreglo

No validar la capacidad provocaba:

`ArrayIndexOutOfBoundsException`

✗ Error 2: Eliminación en cola vacía

Intentar eliminar sin validación generaba comportamientos incorrectos o mensajes erróneos.

✗ Error 3: Mala actualización del índice fin

Incrementar `fin` sin control provocaba posiciones inválidas.

✖ Error 4: Pérdida de datos al recorrer

Un mal ciclo al hacer el desplazamiento podía sobrescribir elementos.

✖ Error 5: Conteo incorrecto

No actualizar correctamente cantidad causaba inconsistencias en el tamaño de la cola.

ArregloNuev

Conclusión

La implementación de una cola mediante arreglos permite comprender de manera clara el funcionamiento interno de esta estructura de datos.

Sin embargo, presenta limitaciones importantes como el tamaño fijo y la necesidad de recorrer elementos al eliminar, lo que la hace menos eficiente en comparación con estructuras dinámicas.

A pesar de esto, es una excelente base para entender conceptos fundamentales de estructuras de dato

5. Punteros

Objetivo

El objetivo de esta implementación es simular el comportamiento de una estructura de datos tipo **cola (FIFO)** utilizando **memoria dinámica mediante punteros en C++**, eliminando las limitaciones de tamaño fijo y permitiendo el crecimiento dinámico de la estructura.

Análisis

A diferencia de los arreglos y `ArrayList`, esta implementación utiliza una **lista enlazada**, donde cada elemento (nodo) contiene:

- Un dato
- Un puntero al siguiente nodo

La cola se controla mediante dos punteros principales:

- **frente:** apunta al primer nodo
- **fin:** apunta al último nodo

Esto permite que las operaciones de inserción y eliminación se realicen de manera eficiente sin necesidad de recorrer o mover elementos.

Implementación

◇ Estructura del Nodo

Se define una estructura básica para representar cada elemento de la cola:

```
struct Nodo {  
    int dato;  
    Nodo* siguiente;  
};
```

◊ Variables principales

- `Nodo* frente;`
- `Nodo* fin;`

Inicialmente ambos apuntan a NULL, indicando que la cola está vacía.

✓ Inserción (agregar)

Proceso:

1. Se crea un nuevo nodo con `new`
2. Si la cola está vacía:
 - a. `frente` y `fin` apuntan al nuevo nodo
3. Si no:
 - a. `fin->siguiente` apunta al nuevo nodo
 - b. Se actualiza `fin`

✓ Eliminación (quitar)

Proceso:

1. Se valida si la cola está vacía
2. Se guarda el nodo del `frente`
3. Se avanza el puntero `frente`
4. Se libera memoria con `delete`

✓ Mostrar

- Se recorre la cola desde `frente` hasta NULL
- Se imprimen los datos de cada nodo

✓ Validaciones

- **`estaVacia():`**
`frente == NULL`

Funcionamiento del Programa

El programa permite al usuario interactuar con la cola mediante un menú que incluye:

1. Insertar elementos
2. Eliminar elementos
3. Mostrar la cola
4. Verificar si está vacía

El flujo se ejecuta en un ciclo hasta que el usuario decide salir.

Resolución de Problemas

✓ Manejo dinámico de memoria

Se utiliza:

- `new` para crear nodos
- `delete` para liberar memoria

✓ Inserción eficiente

No es necesario recorrer la estructura, ya que se utiliza el puntero `fin`.

✓ Eliminación eficiente

Se elimina directamente desde el frente sin mover elementos.

✓ Flexibilidad de tamaño

La cola puede crecer dinámicamente sin límite fijo.

ColaPuntero_DB

Main

```
[*] main.cpp Estructura.h Cola.h
1  #include <iostream>
2  #include "Cola.h"
3
4  using namespace std;
5
6  int main() {
7      Cola c;
8      int opcion;
9
10     do {
11         cout << "\n--- MENU ---\n";
12         cout << "1. Agregar\n2. Quitar\n3. Mostrar\n4. Vacía\n5. Llena\n6. Tamaño\n7. Salir\n";
13         cin >> opcion;
14
15         switch (opcion) {
16             case 1: c.agregar(); break;
17             case 2: c.quitar(); break;
18             case 3: c.mostrar(); break;
19             case 4: cout << (c.estaVacía() ? "Vacía\n" : "No vacía\n"); break;
20             case 5: cout << (c.estaLlena() ? "Llena\n" : "No llena\n"); break;
21             case 6: cout << "Tamaño: " << c.tamaño() << endl; break;
22         }
23
24     } while (opcion != 7);
25
26     return 0;
27 }
```

Clase_Abstracta_PunterosDB

```
[*] main.cpp Estructura.h Cola.h
1  #ifndef ESTRUCTURA_H
2  #define ESTRUCTURA_H
3
4  class Estructura {
5  public:
6      virtual void agregar() = 0;
7      virtual void quitar() = 0;
8      virtual void mostrar() = 0;
9      virtual bool estaVacía() = 0;
10     virtual bool estaLlena() = 0;
11     virtual int tamaño() = 0;
12
13     virtual ~Estructura() {}
14 };
15
16 #endif
```


Clase_Concreta_PunterosDB

```
[*] main.cpp Estructura.h Cola.h
1  #ifndef COLA_H
2  #define COLA_H
3
4  #include <iostream>
5  #include "Estructura.h"
6
7  using namespace std;
8
9  // Nodo con int
10 struct Nodo {
11     int dato;
12     Nodo* siguiente;
13 };
14
15 class Cola : public Estructura {
16 private:
17     Nodo* frente;
18     Nodo* fin;
19     int contador;
20
21 public:
22     Cola() {
23         frente = NULL;
24         fin = NULL;
25         contador = 0;
26     }
27
28     void agregar() override {
29         int valor;
30         cout << "Ingresa un numero: ";
31         cin >> valor;
32
33         Nodo* nuevo = new Nodo;
34         nuevo->dato = valor;
35         nuevo->siguiente = NULL;
36
37         if (estaVacia()) {
38             frente = fin = nuevo;
39         } else {
40             fin->siguiente = nuevo;
41             fin = nuevo;
42         }
43
44         contador++;
45         cout << "Elemento agregado.\n";
46     }
47
48     void quitar() override {
49         if (!estaVacia()) {
50             Nodo* temp = frente;
51             cout << "Elemento eliminado: " << temp->dato << endl;
52
53             frente = frente->siguiente;
54             delete temp;
55             contador--;
56
57             if (frente == NULL)
58                 fin = NULL;
59         } else {
60             cout << "La cola esta vacia.\n";
61         }
62     }
63
64     void mostrar() override {
65         if (estaVacia()) {
66             cout << "La cola esta vacia.\n";
67             return;
68         }
69
70         Nodo* aux = frente;
71         cout << "Contenido: ";
72         while (aux != NULL) {
73             cout << aux->dato << " ";
74             aux = aux->siguiente;
75         }
76         cout << endl;
77     }
78 }
```

```
[*] main.cpp Estructura.h Cola.h
37
38     if (estaVacia()) {
39         frente = fin = nuevo;
40     } else {
41         fin->siguiente = nuevo;
42         fin = nuevo;
43     }
44
45     contador++;
46     cout << "Elemento agregado.\n";
47 }
48
49 void quitar() override {
50     if (!estaVacia()) {
51         Nodo* temp = frente;
52         cout << "Elemento eliminado: " << temp->dato << endl;
53
54         frente = frente->siguiente;
55         delete temp;
56         contador--;
57
58         if (frente == NULL)
59             fin = NULL;
60     } else {
61         cout << "La cola esta vacia.\n";
62     }
63 }
64
65 void mostrar() override {
66     if (estaVacia()) {
67         cout << "La cola esta vacia.\n";
68         return;
69     }
70
71     Nodo* aux = frente;
72     cout << "Contenido: ";
73     while (aux != NULL) {
74         cout << aux->dato << " ";
75         aux = aux->siguiente;
76     }
77     cout << endl;
78 }
```

ColaPuntero_NuevoDato

Main

```
main.cpp Estructura.h [*] Cola.h
1  #include <iostream>
2  #include "Cola.h"
3
4  using namespace std;
5
6  int main() {
7      Cola c;
8      int opcion;
9
10     do {
11         cout << "\n--- MENU ---\n";
12         cout << "1. Agregar persona\n";
13         cout << "2. Quitar persona\n";
14         cout << "3. Mostrar contenido\n";
15         cout << "4. Verificar si esta vacia\n";
16         cout << "5. Verificar si esta llena\n";
17         cout << "6. Mostrar tamaño\n";
18         cout << "7. Salir\n";
19         cout << "Opcion: ";
20         cin >> opcion;
21
22         switch (opcion) {
23             case 1:
24                 c.agregar();
25                 break;
26
27             case 2:
28                 c.quitar();
29                 break;
30
31             case 3:
32                 c.mostrar();
33                 break;
34
35             case 4:
36                 cout << (c.estaVacia() ? "Esta vacia\n" : "No esta vacia\n");
37                 break;
38         }
```

Clase_Abtracta_PunterosND

```
main.cpp Estructura.h [*] Cola.h
1  #ifndef ESTRUCTURA_H
2  #define ESTRUCTURA_H
3
4  class Estructura {
5  public:
6      virtual void agregar() = 0;
7      virtual void quitar() = 0;
8      virtual void mostrar() = 0;
9      virtual bool estaVacia() = 0;
10     virtual bool estaLlena() = 0;
11     virtual int tamano() = 0;
12
13     virtual ~Estructura() {}
14 };
15
16 #endif
```

Clase_Concreta_PunterosND

```
main.cpp Estructura.h [*] Cola.h
1  #ifndef COLA_H
2  #define COLA_H
3
4  #include <iostream>
5  #include <string>
6  #include "Estructura.h"
7
8  using namespace std;
9
10
11 class Persona {
12 public:
13     string nombre;
14     int edad;
15
16     Persona() {
17         nombre = "";
18         edad = 0;
19     }
20
21     Persona(string n, int e) {
22         nombre = n;
23         edad = e;
24     }
25 };
26
27 struct Nodo {
28     Persona dato;
29     Nodo* siguiente;
30 };
31
32
33 class Cola : public Estructura {
34 private:
35     Nodo* frente;
36     Nodo* fin;
37     int contador;
38 }
```

```
32
33 class Cola : public Estructura {
34 private:
35     Nodo* frente;
36     Nodo* fin;
37     int contador;
38
39 public:
40     Cola() {
41         frente = NULL;
42         fin = NULL;
43         contador = 0;
44     }
45
46     void agregar() {
47         string nombre;
48         int edad;
49
50         cout << "Nombre: ";
51         cin >> nombre;
52         cout << "Edad: ";
53         cin >> edad;
54
55         Nodo* nuevo = new Nodo;
56         nuevo->dato = Persona(nombre, edad);
57         nuevo->siguiente = NULL;
58
59         if (estaVacia()) {
60             frente = nuevo;
61             fin = nuevo;
62         } else {
63             fin->siguiente = nuevo;
64             fin = nuevo;
65         }
66
67         contador++;
68         cout << "Persona agregada.\n";
69     }
```

Errores Presentados

✗ Error 1: Fuga de memoria

No liberar memoria con:

```
delete nodo;
```

provoca consumo innecesario de memoria.

✗ Error 2: Acceso a puntero NULL

Intentar acceder a:

```
frente->dato
```

cuando `frente == NULL` causa error.

✗ Error 3: Mala actualización de punteros

No actualizar correctamente `fin` al insertar puede romper la estructura.

✗ Error 4: Cola inconsistente

Eliminar el último nodo sin actualizar `fin` a `NULL`.

5.7 Conclusión

La implementación de una cola mediante punteros en C++ es la más eficiente y flexible de las analizadas.

Permite:

- Manejo dinámico de memoria
- Inserciones y eliminaciones eficientes
- Eliminación de límites de tamaño

Sin embargo, requiere mayor cuidado en el manejo de memoria y punteros, lo que aumenta la complejidad del desarrollo.

6. Librerías

Objetivo

El objetivo de esta implementación es simular el comportamiento de una estructura de datos tipo **cola (FIFO)** utilizando **estructuras dinámicas proporcionadas por librerías**, específicamente `ArrayList`, eliminando las limitaciones de tamaño fijo presentes en los arreglos.

Análisis

A diferencia de la implementación con arreglos, el uso de `ArrayList` permite manejar una estructura dinámica que crece automáticamente conforme se agregan elementos.

En esta implementación:

- No se requiere manejar manualmente índices como `frente` o `fin`
- La librería se encarga de la gestión de memoria
- Las operaciones son más simples y limpias

La cola sigue manteniendo su comportamiento FIFO, pero con una implementación más flexible.

Implementación

La solución se basa en una clase abstracta y una clase concreta.

◇ Clase Abstracta

Define la estructura general de la cola:

- `ArrayList<Integer> datos`
- `int capacidad`

Métodos definidos:

- `agregar()`
- `quitar()`
- `mostrar()`
- `estaVacia()`
- `estaLlena()`
- `tamaño()`

◇ Clase Concreta

Implementa la lógica utilizando los métodos de `ArrayList`.

✓ Inserción (agregar)

- Se valida si la cola está llena (si se definió capacidad)
- Se usa:

```
datos.add(valor);
```

✓ Eliminación (quitar)

- Se valida si la cola está vacía
- Se elimina el primer elemento:

```
datos.remove(0);
```

✓ **Mostrar**

- Se imprime directamente la lista o se recorre:
`for (int i : datos)`

✓ **Validaciones**

- **estaVacía():**
`datos.isEmpty()`
- **estaLlena():**
`datos.size() == capacidad`
- **tamaño():**
`datos.size()`

4.4 Funcionamiento del Programa

El programa presenta un menú interactivo que permite:

1. Agregar elementos
2. Quitar elementos
3. Mostrar contenido
4. Verificar si está vacía
5. Verificar si está llena
6. Consultar tamaño

El sistema se ejecuta en un ciclo hasta que el usuario selecciona salir.

Resolución de Problemas

Durante la implementación se resolvieron los siguientes aspectos:

✓ Manejo automático de memoria

El uso de `ArrayList` elimina la necesidad de controlar manualmente el tamaño del arreglo.

✓ Eliminación eficiente del frente

Aunque internamente `ArrayList` recorre elementos al eliminar el índice 0, el proceso es transparente para el programador.

✓ Simplificación del código

Se redujo la complejidad al no manejar índices manualmente (`frente`, `fin`).

✓ Control de capacidad (opcional)

Se agregó validación para limitar el tamaño máximo si se requiere simular una cola restringida.

Main

Cola_LibreriaDB



The image shows a screenshot of an IDE with three tabs: 'ArreglosDatoBasico\main.java', 'ColaArregloDB_Concreta.java', and 'Aplicaciones_Cola.ir'. The 'main.java' tab is active, displaying the following Java code:

```
1 package Libreria.LibreriaDB;
2
3 import java.util.Scanner;
4
5 public class main {
6     public static void main(String[] args) {
7
8         Scanner sc = new Scanner(System.in);
9
10        System.out.print("Capacidad: ");
11        int cap = sc.nextInt();
12
13        ColaLibreriaDB_Concreta cola = new ColaLibreriaDB_Concreta(cap);
14
15        int op;
16
17        do {
18            System.out.println("\n1. Agregar elemento");
19            System.out.println("2. Quitar elemento");
20            System.out.println("3. Mostrar");
21            System.out.println("4. ¿Está vacía?");
22            System.out.println("5. ¿Está llena?");
23            System.out.println("6. Tamaño");
24            System.out.println("7. Salir");
25            System.out.print("Opción: ");
26
27            op = sc.nextInt();
28
29            switch (op) {
30                case 1:
31                    System.out.print("Valor: ");
32                    cola.agregar(sc.nextInt());
33                    break;
```

ColaLibreriaDB_Abstracta

```
main.java ColaLibreriaDB_Abstracta.java ColaArregloDB_Abstracta.java

1 package Libreria.LibreriaDB;
2
3 import java.util.ArrayList;
4
5 public abstract class ColaLibreriaDB_Abstracta { 1 usage 1 inheritor
6     protected ArrayList<Integer> datos; 7 usages
7     protected int capacidad;
8
9     public ColaLibreriaDB_Abstracta(int capacidad) { 1 usage
10         this.capacidad = capacidad;
11         datos = new ArrayList<>();
12     }
13
14     public abstract void agregar(int valor); 1 implementation
15     public abstract void quitar(); 1 implementation
16     public abstract void mostrar(); 1 implementation
17     public abstract boolean estaVacia(); 1 implementation
18     public abstract boolean estaLlena(); 1 implementation
19     public abstract int tamano(); 1 usage 1 implementation
20 }
```

ColaLibreriaDB_Concreta

```
main.java ColaLibreriaDB_Abstracta.java ColaLibreriaDB_Concreta.java
1 package Libreria.LibreriaDB;
2
3 public class ColaLibreriaDB_Concreta extends ColaLibreriaDB_Abstracta { 2 usages
4     public ColaLibreriaDB_Concreta(int capacidad) { super(capacidad); }
5
6     @Override
7     public void agregar(int valor) {
8         if (estaLlena()) {
9             System.out.println("La cola está llena.");
10        } else {
11            datos.add(valor); // FIFO
12        }
13    }
14
15    @Override
16    public void quitar() {
17        if (estaVacia()) {
18            System.out.println("La cola está vacía.");
19        } else {
20            System.out.println("Eliminado: " + datos.remove(index: 0));
21        }
22    }
23
24    @Override
25    public void mostrar() {
26        if (estaVacia()) {
27            System.out.println("Vacía");
28        } else {
29            System.out.println(datos);
30        }
31    }
32
33    @Override
34    public boolean estaVacia() { return datos.isEmpty(); }
```

```

35    @Override
36    public boolean estaLlena() { return datos.size() == capacidad; }
37
38    @Override 1 usage
39    public int tamano() { return datos.size(); }
40 }
```

ColaLibreriaND

Main

```
LibreriaDB\main.java ColaLibreriaDB_Abstracta.java ColaLibreriaDB_Concreta.java
1 package Libreria.LibreriaNuevoDato;
2
3 > import ...
4
5
6
7 public class main {
8     public static void main(String[] args) {
9
10         Scanner sc = new Scanner(System.in);
11
12         System.out.print("Capacidad: ");
13         int cap = sc.nextInt();
14         sc.nextLine();
15
16         ColaLibreriaND_Concreta cola = new ColaLibreriaND_Concreta(cap);
17
18         int op;
19
20         do {
21             System.out.println("\n1. Agregar producto");
22             System.out.println("2. Quitar producto");
23             System.out.println("3. Mostrar");
24             System.out.println("4. ¿Está vacía?");
25             System.out.println("5. ¿Está llena?");
26             System.out.println("6. Tamaño");
27             System.out.println("7. Salir");
28             System.out.print("Opción: ");
29
30             op = sc.nextInt();
31             sc.nextLine();
32
33             switch (op) {
34                 case 1:
35                     System.out.print("Nombre: ");
```

Cola_LibreriaND_Annstracta

```
loads\Proyect_2-main Libreria.LibreriaNuevoDato;  
2  
3 import java.util.ArrayList;  
4  
5 public abstract class ColaLibreriaND_Abstracta { 1 usage 1 inheritor  
6  
7     protected ArrayList<Producto> datos; 7 usages  
8     protected int capacidad;  
9  
10    public ColaLibreriaND_Abstracta(int capacidad) { 1 usage  
11        this.capacidad = capacidad;  
12        datos = new ArrayList<>();  
13    }  
14  
15    public abstract void agregar(Producto p); 1 implementation  
16    public abstract void quitar(); 1 implementation  
17    public abstract void mostrar(); 1 implementation  
18    public abstract boolean estaVacía(); 1 implementation  
19    public abstract boolean estaLlena(); 1 implementation  
20    public abstract int tamaño(); 1 usage 1 implementation  
21 }
```

ColaLibreriaND_Concreta

```
main.java x ColaLibreriaND_Abstracta.java ColaLibreriaND_Concreta.java x  
1 package Libreria.LibreriaNuevoDato;  
2  
3 public class ColaLibreriaND_Concreta extends ColaLibreriaND_Abstracta  
4  
5     public ColaLibreriaND_Concreta(int capacidad) { super(capacidad);  
6  
7  
8  
9  
10    @Override  
11    public void agregar(Producto p) {  
12        if (estaLlena()) {  
13            System.out.println("La cola está llena.");  
14        } else {  
15            datos.add(p);  
16        }  
17    }  
18  
19    @Override  
20    public void quitar() {  
21        if (estaVacía()) {  
22            System.out.println("La cola está vacía.");  
23        } else {  
24            System.out.println("Eliminado: " + datos.remove(index: 0));  
25        }  
26    }  
27  
28    @Override  
29    public void mostrar() {  
30        if (estaVacía()) {  
31            System.out.println("Vacía");  
32        } else {  
33            for (Producto p : datos) {  
34                System.out.println(p);  
35            }  
36        }  
37    }  
38  
39    @Override  
40    public boolean estaVacía() { return datos.isEmpty(); }  
41  
42    @Override  
43    public boolean estaLlena() { return datos.size() == capacidad; }  
44  
45    @Override 1 usage  
46    public int tamaño() { return datos.size(); }  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000  
1001  
1002  
1003  
1004  
1005  
1006  
1007  
1008  
1009  
1010  
1011  
1012  
1013  
1014  
1015  
1016  
1017  
1018  
1019  
1020  
1021  
1022  
1023  
1024  
1025  
1026  
1027  
1028  
1029  
1030  
1031  
1032  
1033  
1034  
1035  
1036  
1037  
1038  
1039  
1040  
1041  
1042  
1043  
1044  
1045  
1046  
1047  
1048  
1049  
1050  
1051  
1052  
1053  
1054  
1055  
1056  
1057  
1058  
1059  
1060  
1061  
1062  
1063  
1064  
1065  
1066  
1067  
1068  
1069  
1070  
1071  
1072  
1073  
1074  
1075  
1076  
1077  
1078  
1079  
1080  
1081  
1082  
1083  
1084  
1085  
1086  
1087  
1088  
1089  
1090  
1091  
1092  
1093  
1094  
1095  
1096  
1097  
1098  
1099  
1100  
1101  
1102  
1103  
1104  
1105  
1106  
1107  
1108  
1109  
1110  
1111  
1112  
1113  
1114  
1115  
1116  
1117  
1118  
1119  
1120  
1121  
1122  
1123  
1124  
1125  
1126  
1127  
1128  
1129  
1130  
1131  
1132  
1133  
1134  
1135  
1136  
1137  
1138  
1139  
1140  
1141  
1142  
1143  
1144  
1145  
1146  
1147  
1148  
1149  
1150  
1151  
1152  
1153  
1154  
1155  
1156  
1157  
1158  
1159  
1160  
1161  
1162  
1163  
1164  
1165  
1166  
1167  
1168  
1169  
1170  
1171  
1172  
1173  
1174  
1175  
1176  
1177  
1178  
1179  
1180  
1181  
1182  
1183  
1184  
1185  
1186  
1187  
1188  
1189  
1190  
1191  
1192  
1193  
1194  
1195  
1196  
1197  
1198  
1199  
1200  
1201  
1202  
1203  
1204  
1205  
1206  
1207  
1208  
1209  
1210  
1211  
1212  
1213  
1214  
1215  
1216  
1217  
1218  
1219  
1220  
1221  
1222  
1223  
1224  
1225  
1226  
1227  
1228  
1229  
1230  
1231  
1232  
1233  
1234  
1235  
1236  
1237  
1238  
1239  
1240  
1241  
1242  
1243  
1244  
1245  
1246  
1247  
1248  
1249  
1250  
1251  
1252  
1253  
1254  
1255  
1256  
1257  
1258  
1259  
1260  
1261  
1262  
1263  
1264  
1265  
1266  
1267  
1268  
1269  
1270  
1271  
1272  
1273  
1274  
1275  
1276  
1277  
1278  
1279  
1280  
1281  
1282  
1283  
1284  
1285  
1286  
1287  
1288  
1289  
1290  
1291  
1292  
1293  
1294  
1295  
1296  
1297  
1298  
1299  
1300  
1301  
1302  
1303  
1304  
1305  
1306  
1307  
1308  
1309  
1310  
1311  
1312  
1313  
1314  
1315  
1316  
1317  
1318  
1319  
1320  
1321  
1322  
1323  
1324  
1325  
1326  
1327  
1328  
1329  
1330  
1331  
1332  
1333  
1334  
1335  
1336  
1337  
1338  
1339  
1340  
1341  
1342  
1343  
1344  
1345  
1346  
1347  
1348  
1349  
1350  
1351  
1352  
1353  
1354  
1355  
1356  
1357  
1358  
1359  
1360  
1361  
1362  
1363  
1364  
1365  
1366  
1367  
1368  
1369  
1370  
1371  
1372  
1373  
1374  
1375  
1376  
1377  
1378  
1379  
1380  
1381  
1382  
1383  
1384  
1385  
1386  
1387  
1388  
1389  
1390  
1391  
1392  
1393  
1394  
1395  
1396  
1397  
1398  
1399  
1400  
1401  
1402  
1403  
1404  
1405  
1406  
1407  
1408  
1409  
1410  
1411  
1412  
1413  
1414  
1415  
1416  
1417  
1418  
1419  
1420  
1421  
1422  
1423  
1424  
1425  
1426  
1427  
1428  
1429  
1430  
1431  
1432  
1433  
1434  
1435  
1436  
1437  
1438  
1439  
1440  
1441  
1442  
1443  
1444  
1445  
1446  
1447  
1448  
1449  
1450  
1451  
1452  
1453  
1454  
1455  
1456  
1457  
1458  
1459  
1460  
1461  
1462  
1463  
1464  
1465  
1466  
1467  
1468  
1469  
1470  
1471  
1472  
1473  
1474  
1475  
1476  
1477  
1478  
1479  
1480  
1481  
1482  
1483  
1484  
1485  
1486  
1487  
1488  
1489  
1490  
1491  
1492  
1493  
1494  
1495  
1496  
1497  
1498  
1499  
1500  
1501  
1502  
1503  
1504  
1505  
1506  
1507  
1508  
1509  
1510  
1511  
1512  
1513  
1514  
1515  
1516  
1517  
1518  
1519  
1520  
1521  
1522  
1523  
1524  
1525  
1526  
1527  
1528  
1529  
1530  
1531  
1532  
1533  
1534  
1535  
1536  
1537  
1538  
1539  
1540  
1541  
1542  
1543  
1544  
1545  
1546  
1547  
1548  
1549  
1550  
1551  
1552  
1553  
1554  
1555  
1556  
1557  
1558  
1559  
1560  
1561  
1562  
1563  
1564  
1565  
1566  
1567  
1568  
1569  
1570  
1571  
1572  
1573  
1574  
1575  
1576  
1577  
1578  
1579  
1580  
1581  
1582  
1583  
1584  
1585  
1586  
1587  
1588  
1589  
1590  
1591  
1592  
1593  
1594  
1595  
1596  
1597  
1598  
1599  
1600  
1601  
1602  
1603  
1604  
1605  
1606  
1607  
1608  
1609  
1610  
1611  
1612  
1613  
1614  
1615  
1616  
1617  
1618  
1619  
1620  
1621  
1622  
1623  
1624  
1625  
1626  
1627  
1628  
1629  
1630  
1631  
1632  
1633  
1634  
1635  
1636  
1637  
1638  
1639  
1640  
1641  
1642  
1643  
1644  
1645  
1646  
1647  
1648  
1649  
1650  
1651  
1652  
1653  
1654  
1655  
1656  
1657  
1658  
1659  
1660  
1661  
1662  
1663  
1664  
1665  
1666  
1667  
1668  
1669  
1670  
1671  
1672  
1673  
1674  
1675  
1676  
1677  
1678  
1679  
1680  
1681  
1682  
1683  
1684  
1685  
1686  
1687  
1688  
1689  
1690  
1691  
1692  
1693  
1694  
1695  
1696  
1697  
1698  
1699  
1700  
1701  
1702  
1703  
1704  
1705  
1706  
1707  
1708  
1709  
1710  
1711  
1712  
1713  
1714  
1715  
1716  
1717  
1718  
1719  
1720  
1721  
1722  
1723  
1724  
1725  
1726  
1727  
1728  
1729  
1730  
1731  
1732  
1733  
1734  
1735  
1736  
1737  
1738  
1739  
1740  
1741  
1742  
1743  
1744  
1745  
1746  
1747  
1748  
1749  
1750  
1751  
1752  
1753  
1754  
1755  
1756  
1757  
1758  
1759  
1760  
1761  
1762  
1763  
1764  
1765  
1766  
1767  
1768  
1769  
1770  
1771  
1772  
1773  
1774  
1775  
1776  
1777  
1778  
1779  
1780  
1781  
1782  
1783  
1784  
1785  
1786  
1787  
1788  
1789  
1790  
1791  
1792  
1793  
1794  
1795  
1796  
1797  
1798  
1799  
1800  
1801  
1802  
1803  
1804  
1805  
1806  
1807  
1808  
1809  
1810  
1811  
1812  
1813  
1814  
1815  
1816  
1817  
1818  
1819  
1820  
1821  
1822  
1823  
1824  
1825  
1826  
1827  
1828  
1829  
1830  
1831  
1832  
1833  
1834  
1835  
1836  
1837  
1838  
1839  
1840  
1841  
1842  
1843  
1844  
1845  
1846  
1847  
1848  
1849  
1850  
1851  
1852  
1853  
1854  
1855  
1856  
1857  
1858  
1859  
1860  
1861  
1862  
1863  
1864  
1865  
1866  
1867  
1868  
1869  
1870  
1871  
1872  
1873  
1874  
1875  
1876  
1877  
1878  
1879  
1880  
1881  
1882  
1883  
1884  
1885  
1886  
1887  
1888  
1889  
1890  
1891  
1892  
1893  
1894  
1895  
1896  
1897  
1898  
1899  
1900  
1901  
1902  
1903  
1904  
1905  
1906  
1907  
1908  
1909  
1910  
1911  
1912  
1913  
1914  
1915  
1916  
1917  
1918  
1919  
1920  
1921  
1922  
1923  
1924  
1925  
1926  
1927  
1928  
1929  
1930  
1931  
1932  
1933  
1934  
1935  
1936  
1937  
1938  
1939  
1940  
1941  
1942  
1943  
1944  
1945  
1946  
1947  
1948  
1949  
1950  
1951  
1952  
1953  
1954  
1955  
1956  
1957  
1958  
1959  
1960  
1961  
1962  
1963  
1964  
1965  
1966  
1967  
1968  
1969  
1970  
1971  
1972  
1973  
1974  
1975  
1976  
1977  
1978  
1979  
1980  
1981  
1982  
1983  
1984  
1985  
1986  
1987  
1988  
1989  
1990  
1991  
1992  
1993  
1994  
1995  
1996  
1997  
1998  
1999  
2000  
2001  
2002  
2003  
2004  
2005  
2006  
2007  
2008  
2009  
2010  
2011  
2012  
2013  
2014  
2015  
2016  
2017  
2018  
2019  
2020  
2021  
2022  
2023  
2024  
2025  
2026  
2027  
2028  
2029  
2030  
2031  
2032  
2033  
2034  
2035  
2036  
2037  
2038  
2039  
2040  
2041  
2042  
2043  
2044  
2045  
2046  
2047  
2048  
2049  
2050  
2051  
2052  
2053  
2054  
2055  
2056  
2057  
2058  
2059  
2060  
2061  
2062  
2063  
2064  
2065  
2066  
2067  
2068  
2069  
2070  
2071  
2072  
2073  
2074  
2075  
2076  
2077  
2078  
2079  
2080  
2081  
2082  
2083  
2084  
2085  
2086  
2087  
2088  
2089  
2090  
2091  
2092  
2093  
2094  
2095  
2096  
2097  
2098  
2099  
2100  
2101  
2102  
2103  
2104  
2105  
2106  
2107  
2108  
2109  
2110  
2111  
2112  
2113  
2114  
2115  
2116  
2117  
2118  
2119  
2120  
2121  
2122  
2123  
2124  
2125  
2126  
2127  
2128  
2129  
2130  
2131  
2132  
2133  
21
```

Errores Presentados

✗ Error 1: Eliminación en lista vacía

Intentar ejecutar:

```
datos.remove(0);
```

sin validar puede provocar:

`IndexOutOfBoundsException`

✗ Error 2: No validar capacidad

Si se definió un límite y no se valida:

- La cola puede crecer sin control

✗ Error 3: Uso incorrecto de tipos

Intentar insertar objetos en:

```
ArrayList<Integer>
```

provoca error de tipo

✗ Error 4: Confusión con referencias

Modificar objetos dentro del `ArrayList` puede afectar otros datos si se reutilizan referencias.

✗ Error 5: Iteración incorrecta

Recorrer mal la lista puede causar errores lógicos o mostrar datos incompletos.

Conclusión

La implementación de una cola mediante `ArrayList` ofrece una solución más flexible y sencilla en comparación con los arreglos.

Permite:

- Manejo dinámico de datos
- Código más limpio
- Menor probabilidad de errores relacionados con índices

Sin embargo, pierde cierto control sobre la memoria y puede tener un pequeño costo en rendimiento al eliminar elementos del inicio.

IV. LISTAS

7. Array

El objetivo de este programa es simular el comportamiento de una estructura de datos tipo "Lista" utilizando un arreglo estático de tamaño fijo. A diferencia de una Pila (LIFO) o una Cola (FIFO), una Lista pura nos permite mayor flexibilidad, como consultar, insertar o eliminar elementos en posiciones específicas o de forma secuencial.

Para mantener el orden, la solución se estructuró de la siguiente manera:

- **Persona.java (Solo para la fase 2):** Clase que define nuestro Nuevo Tipo de Dato con sus atributos (nombre y edad).
- **ListaEstatica.java:** Clase encargada de administrar el arreglo. Contiene el arreglo interno (con un límite máximo predefinido), una variable cantidad (para llevar el control de cuántos espacios están ocupados) y los métodos meterValorPosicion(), eliminaValorPosicion(), y mostrar().
- **Main.java:** Clase principal donde se despliega un menú interactivo para probar las funciones de la lista, tanto para los números como para los objetos.

La clave para administrar una lista estática recae totalmente en la variable contadora (que llamamos cantidad o tope). Como un arreglo de 10 posiciones nace con los 10 espacios físicos ya creados en la RAM, no podemos simplemente "destruir" un cajón.

- Para insertar: Verificamos que cantidad sea menor al tamaño máximo del arreglo para evitar un desbordamiento. Si hay espacio, colocamos el dato en arreglo[cantidad] y sumamos 1 a nuestro contador.
- Para eliminar: Si decidimos sacar un elemento del medio de la lista, la lógica nos obliga a hacer un desplazamiento (shift). Es decir, utilizamos un ciclo for para mover todos los elementos que estaban a la derecha del hueco un paso hacia la izquierda, y finalmente restamos 1 a nuestra variable cantidad.

CABE ACLARAR QUE EL MAIN Y PERSONA SERIA PRACTICAMENTE LO MISMO PARA ARRAY, Y LIBRERÍA, POR LO QUE SOLO SE PONDRA UN MAIN Y PERSONA EN EL ARRAY COMO SIRVIRA COMO REFERENCIA PARA EL DE LIBRERIA EN (DB Y NUEVO TIPO DE DATO)

Dato Básico (DB)

main.java

```
1  import java.util.Scanner;
2
3  public class main {
4      public static void main(String[] args) {
5          Scanner lector = new Scanner(System.in);
6          ADTArrayDB misNumeros = new ListaArrayBasica( capacidad: 7);
7          int cantidadAInsertar;
8          int op;
9
10         do{
11             System.out.println("<===Menu===>");
12             System.out.println("1. Agregar elemento \n" +
13                 "2. Eliminar elemento \n" +
14                 "3. Mostrar contenido \n" +
15                 "4. Esta la lista llena? \n" +
16                 "5. Esta la lista vacia? \n" +
17                 "6. Salir");
18             op = lector.nextInt();
19
20             switch (op){
21
22                 case 1:
23                     int espaciosLibres = 7 - misNumeros.getTamActual();
24
25                     if (espaciosLibres == 0) {
26                         System.out.println("La lista ya se lleno, requieres eliminar elementos antes \n");
27                         break;
28                     }
29                     System.out.println("Cuantos valores deseas meter? (MAX 7)");
30                     cantidadAInsertar = lector.nextInt();
31
32                     if (cantidadAInsertar <= espaciosLibres && cantidadAInsertar > 0){
33                         System.out.println("=====METER VALORES=====");
34                         for (int i = 0; i < cantidadAInsertar; i++) {
35                             System.out.println("\n--- Ingreso " + (i + 1) + " ---");
36                             System.out.print("Ingresar un numero entero: ");
37                             int num = lector.nextInt();
38                             if (misNumeros.insertarOrdenado(num)) {
39                                 System.out.println("Se ingreso correctamente el numero");
40                             } else {
41                                 System.out.println("La lista esta llena");
42                             }
43                         }
44                     }else{
45                         System.out.println("No se pueden guardar esa cantidad, solo tienes de espacio: " + espaciosLibres + " espacios.");
46                     }
47                     break;
48
49                 case 2:
50                     if (misNumeros.getTamActual() == 0){
51                         System.out.println("No se puede eliminar nada, no hay ningún elemento");
52                     }else {
53                         System.out.println("\n===== ELIMINAR POR VALOR =====");
54                         System.out.print("¿Qué número deseas eliminar de la lista? ");
55                         int numEliminar = lector.nextInt();
56
57                         if (misNumeros.eliminarPorValor(numEliminar)) {
58                             System.out.println("El numero " + numEliminar + " ha sido eliminado");
59                         } else {
60                             System.out.println("El numero " + numEliminar + " no se encuentra en la lista.");
61                         }
62                     }
63                     break;
64
65                 case 3:
66                     System.out.println("Mostrando lista... \n");
67                     misNumeros.mostrar();
68                     break;
69                 case 4:
70                     misNumeros.llena();
71                     break;
72                 case 5:
73                     misNumeros.vacia();
74                     break;
75                 case 6:
76                     System.out.println("Saliendo del menu...");
77                     break;
78                 default:
79                     System.out.println("\nOpcion no valida, seleccionar las opciones que aparecen en el menu");
80                     break;
81             }
82         } while (op != 6);
83     }
84 }
85 }
```


ListaArrayBasica.java

```

1
2 public class ListaArrayBasica extends ADTArrayDB { 1 usage
3     private int[] elementos; 9 usages
4     private int tamActual; 16 usages
5     private final int MAX; 3 usages
6
7     public ListaArrayBasica(int capacidad) { 1 usage
8         this.MAX = capacidad;
9         this.elementos = new int[MAX];
10        this.tamActual = 0;
11    }
12
13    @Override 1 usage
14    public void mostrar() {
15        if (tamActual == 0) {
16            System.out.println("Lista vacia.");
17            return;
18        }
19        System.out.print("Lista: ");
20        for (int i = 0; i < tamActual; i++) {
21            System.out.print(elementos[i] + " ");
22        }
23        System.out.println();
24    }
25
26    @Override 1 usage
27    public boolean insertarOrdenado(int numero) {
28        if (tamActual >= MAX) return false;
29
30        int pos = 0;
31        while (pos < tamActual && elementos[pos] < numero) {
32            pos++;
33        }

```

Consola

```

0. Salir
3
Mostrando lista...

Lista: 12 23 23 32
<==Menu==>
1. Agregar elemento
2. Eliminar elemento
3. Mostrar contendio
4. Esta la lista llena?
5. Esta la lista vacia?
6. Salir
2

===== ELIMINAR POR VALOR =====
¿Qué número deseas eliminar de la lista? 23
El numero 23 ha sido eliminado
<==Menu==>
1. Agregar elemento
2. Eliminar elemento
3. Mostrar contendio
4. Esta la lista llena?
5. Esta la lista vacia?
6. Salir
3
Mostrando lista...

Lista: 12 23 32

```

ADTArrayDB.java

```

1
2 public abstract class ADTArrayDB { 2 usages 1 inheritor
3     public abstract boolean insertarOrdenado(int numero);
4     public abstract boolean eliminarPorValor(int numero);
5     public abstract void mostrar(); 1 usage 1 implementation
6     public abstract void llena(); 1 usage 1 implementation
7     public abstract void vacia(); 1 usage 1 implementation
8     public abstract int getTamActual(); 2 usages 1 implementa
9 }

```

Nuevo Tipo de Dato

ListaArray.java

```
11 public void mostrar() {
12     if (tamActual == 0) {
13         System.out.println("Lista vacia.");
14         return;
15     }
16     System.out.println("Lista de Contactos:");
17     for (int i = 0; i < tamActual; i++) {
18         System.out.println(elementos[i].toString());
19     }
20     System.out.println();
21 }
22 @Override 1usage
23 public boolean meterValorPosicion(Persona p) {
24     if (tamActual >= MAX) return false;
25     int pos = 0;
26     while (pos < tamActual && elementos[pos].edad < p.edad) {
27         pos++;
28     }
29     for (int i = tamActual; i > pos; i--) {
30         elementos[i] = elementos[i - 1];
31     }
32     elementos[pos] = p;
33     tamActual++;
34     return true;
35 }
36
37 public void estaLlena() {
38     if (tamActual == 7) {
39         System.out.println("Esta llena la lista");
40     } else {
41         int disponibles = 7 - tamActual;
42         System.out.println("No esta llena, espacios disponibles: " + disponibles);
43     }
44 }
45
46 @Override 1usage
47 public void estaVacua() {
48     if (tamActual == 0) {
49         System.out.println("Esta vacia la lista");
50     } else {
51         System.out.println("El numero de datos guardados: " + tamActual);
52     }
53 }
54
55 @Override 2usages
56 public int getTamActual() { return tamActual; }
```

Consola

```
Lista de Contactos:
Persona {Nombre: pepe, Edad: 18}
Persona {Nombre: jorge, Edad: 19}
```

```
<===Menu===>
```

1. Agregar elemento
2. Eliminar elemento
3. Mostrar contendio
4. Esta la lista llena?
5. Esta la lista vacia?
6. Salir

```
2
```

```
===== ELIMINAR POR NOMBRE =====
```

```
Que nombre deseas eliminar de la lista? pepe
El registro de pepe ha sido eliminado.
```

```
<===Menu===>
```

1. Agregar elemento
2. Eliminar elemento
3. Mostrar contendio
4. Esta la lista llena?
5. Esta la lista vacia?
6. Salir

```
3
```

```
Mostrando lista...
```

```
Lista de Contactos:
Persona {Nombre: jorge, Edad: 19}
```

ADTListNTD.java

```
1 public abstract class ADTListNTD { 4 usages 1inheritor
2     public abstract boolean meterValorPosicion(Persona p); 1usage
3     public abstract boolean eliminaValorPosicion(String nombre);
4     public abstract void mostrar(); 1usage 1implementation
5     public abstract void estaLlena(); 1usage 1implementation
6     public abstract void estaVacua(); 1usage 1implementation
7     public abstract int getTamActual(); 2usages 1implementation
8 }
```

Hubo los siguientes errores, que se presentaron al hacer el código;

Error de Desbordamiento (ArrayIndexOutOfBoundsException):

- *El problema:* En las primeras pruebas, si el usuario intentaba meter 11 elementos en una lista diseñada para 10, el programa colapsaba inmediatamente.

La solución: Se implementó programación defensiva. Se agregó una validación if (cantidad < arreglo.length) al principio del método insertar(). Si el arreglo está lleno, se bloquea la inserción y se le muestra un mensaje amigable al usuario indicando que "La lista está llena", protegiendo la ejecución del programa.

El problema del "hueco Fantasma" al eliminar:

- *El problema:* Al eliminar un dato (especialmente en el Nuevo Tipo de Dato), si solo poníamos el espacio en null o 0, la lista quedaba fragmentada (ej. [Carlos, Ana, null, Pedro]). Esto rompía el método mostrar() y desperdiciaba espacio.

La solución: Se programó un algoritmo de recorrido inverso. Al eliminar a "Ana", el código toma a "Pedro" y lo recorre una posición hacia la izquierda para tapar el hueco. Para la versión de Objetos, además de recorrer los elementos, nos aseguramos de igualar la última posición repetida a null para liberar la referencia y tener un código limpio de basura.

Como se puede ver, el código es muy similar en ambos casos, para dato básico y para nuevo tipo de dato, sin embargo, en uno pedimos números, y se hacen pequeños cambios en la clase ListaEstaticaBasica.java, tampoco ocupamos la clase Persona.java debido que no manejamos datos como Nombre, y Edad, y es lo que hace la diferencia, en dato básico no se depende de otra clase, y solo trabaja con los métodos que se hayan creado (POO) y se usan valores como ya mencionado, números, como lo es Int, Float, etc...

8. Puntero

El objetivo de este programa es superar la limitación principal de los arreglos estáticos: el tamaño fijo. Mediante el uso de memoria dinámica y punteros (en lenguaje C++), construimos una Lista que puede crecer o encogerse en tiempo de ejecución de manera infinita (limitada solo por la memoria RAM física). En lugar de usar un bloque continuo de memoria (como un arreglo), usamos "Nodos" esparcidos en la memoria que se toman de la mano apuntando al siguiente elemento. Al igual que en el ejercicio anterior, este concepto se implementó en dos fases para demostrar su dominio, en **dato básico** se almacena en cada nodo un numero entero simple, en **Nuevo tipo de dato** cada nodo almacena un struct Persona o un objeto complejo

Se estructuró dividiendo las declaraciones (.h) de las implementaciones (.cpp):

- **Nodo:** La estructura fundamental. Actúa como un contenedor que guarda el dato (ya sea el int o la Persona) y un puntero *siguiente que guarda la dirección de memoria del próximo Nodo de la lista.
- **ListaDinamica (y ListaPersonas):** La clase administradora. Su único atributo físico es un puntero llamado cabeza (o inicio). Contiene los métodos para pedir memoria (new) al insertar(), y liberarla (delete) al eliminar().
- **Main.cpp:** Archivo de ejecución con el menú interactivo para el usuario.

La lógica de los punteros se basa en "enganchar y desenganchar" direcciones de memoria sin perder la referencia de la cabeza.

- Al insertar, el programa solicita una nueva "caja" a la RAM mediante la palabra reservada new Nodo(). El puntero siguiente de este nuevo Nodo se iguala a la cabeza actual, y luego la cabeza se mueve para apuntar al recién llegado.
- Al mostrar o buscar, se crea un puntero temporal (ej. actual) que funciona como un "explorador". Recorre la lista saltando de actual = actual->siguiente hasta llegar a nullptr (el final).

Dato Básico (DB)

ListaDinamica.h

```
1  #ifndef LISTADINAMICA_H
2  #define LISTADINAMICA_H
3  #include <iostream>
4  using namespace std;
5  class Nodo {
6  public:
7      int dato;
8      Nodo* siguiente;
9  };
10 class ListaDinamica {
11 private:
12     Nodo* cabeza;
13     int tamActual;
14 public:
15     ListaDinamica();
16     ~ListaDinamica();
17
18     bool insertarOrdenado(int valor);
19     bool eliminarPorValor(int valor);
20     void mostrar();
21     bool estaVacia();
22     int getTamActual();
23 };
24 #endif
```

ListaDinamica.cpp

```
1  #include "ListaDinamica.h"
2  ListaDinamica::ListaDinamica() {
3      cabeza = nullptr;
4      tamActual = 0;
5  }
6  ListaDinamica::~ListaDinamica() {
7      Nodo* actual = cabeza;
8      while (actual != nullptr) {
9          Nodo* siguiente = actual->siguiente;
10         delete actual;
11         actual = siguiente;
12     }
13     cout << "Memoria RAM liberada correctamente al cerrar el programa." << endl;
14 }
15 bool ListaDinamica::insertarOrdenado(int valor) {
16     Nodo* nuevoNodo = new Nodo();
17     nuevoNodo->dato = valor;
18     nuevoNodo->siguiente = nullptr;
19     if (cabeza == nullptr || cabeza->dato >= valor) {
20         nuevoNodo->siguiente = cabeza;
21         cabeza = nuevoNodo;
22     }
23     else {
24         Nodo* actual = cabeza;
25         while (actual->siguiente != nullptr && actual->siguiente->dato < valor) {
26             actual = actual->siguiente;
27         }
28         nuevoNodo->siguiente = actual->siguiente;
29         actual->siguiente = nuevoNodo;
30     }
31     tamActual++;
32     return true;
33 }
```

```

33     }
34     bool ListaDinamica::eliminarPorValor(int valor) {
35         if (cabeza == nullptr) return false;
36         if (cabeza->dato == valor) {
37             Nodo* nodoABorrar = cabeza;
38             cabeza = cabeza->siguiente;
39             delete nodoABorrar;
40             tamActual--;
41             return true;
42         }
43         Nodo* actual = cabeza;
44         while (actual->siguiente != nullptr && actual->siguiente->dato != valor) {
45             actual = actual->siguiente;
46         }
47         if (actual->siguiente != nullptr) {
48             Nodo* nodoABorrar = actual->siguiente;
49             actual->siguiente = nodoABorrar->siguiente;
50             delete nodoABorrar;
51             tamActual--;
52             return true;
53         }
54         return false;
55     }

56     void ListaDinamica::mostrar() {
57         if (cabeza == nullptr) {
58             cout << "Lista vacia." << endl;
59             return;
60         }
61         Nodo* actual = cabeza;
62         cout << "Lista: ";
63         while (actual != nullptr) {
64             cout << "[" << actual->dato << "]" -> ";";
65             actual = actual->siguiente;
66         }
67         cout << "NULL\n" << endl;
68     }
69     bool ListaDinamica::estaVacia() {
70         return cabeza == nullptr;
71     }
72     int ListaDinamica::getTamActual() {
73         return tamActual;
74     }

```

Consola

--- Ingreso 2 ---

Ingresa un numero entero: 12
Se ingreso correctamente el numero.

<=== Menu ===>

1. Agregar elemento
2. Eliminar elemento por valor
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir

Elige una opcion: 3

Mostrando lista...

Lista: [12] -> [23] -> NULL

<=== Menu ===>

1. Agregar elemento
2. Eliminar elemento por valor
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir

Elige una opcion: 2

===== ELIMINAR POR VALOR =====

Que numero deseas eliminar de la lista? 12

El numero 12 ha sido eliminado y la memoria liberada.

<=== Menu ===>

1. Agregar elemento
2. Eliminar elemento por valor
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir

Elige una opcion: 3

Mostrando lista...

Lista: [23] -> NULL

Nuevo Tipo de Dato

ListaPersonas.cpp

```
1  #include "ListaPersonas.h"
2  ListaPersonas::ListaPersonas() {
3      cabeza = nullptr;
4      tamActual = 0;
5  }
6  ListaPersonas::~ListaPersonas() {
7      Nodo* actual = cabeza;
8      while (actual != nullptr) {
9          Nodo* siguiente = actual->siguiente;
10         delete actual;
11         actual = siguiente;
12     }
13     cout << "Toda la memoria de la lista ha sido liberada correctamente." << endl;
14 }
15 bool ListaPersonas::insertarOrdenado(string nom, int ed) {
16     Nodo* nuevoNodo = new Nodo();
17     nuevoNodo->dato.nombre = nom;
18     nuevoNodo->dato.edad = ed;
19     nuevoNodo->siguiente = nullptr;
20     if (cabeza == nullptr || cabeza->dato.edad >= ed) {
21         nuevoNodo->siguiente = cabeza;
22         cabeza = nuevoNodo;
23     }
24     else {
25         Nodo* actual = cabeza;
26
27         while (actual->siguiente != nullptr && actual->siguiente->dato.edad < ed) {
28             actual = actual->siguiente;
29         }
30         nuevoNodo->siguiente = actual->siguiente;
31         actual->siguiente = nuevoNodo;
32     }
33     tamActual++;
34     return true;
35 }
36 bool ListaPersonas::eliminarPorNombre(string nom) {
37     if (cabeza == nullptr) return false;
38     if (cabeza->dato.nombre == nom) {
39         Nodo* nodoABorrar = cabeza;
40         cabeza = cabeza->siguiente;
41         delete nodoABorrar;
42         tamActual--;
43         return true;
44     }
45     Nodo* actual = cabeza;
46     while (actual->siguiente != nullptr && actual->siguiente->dato.nombre != nom) {
47         actual = actual->siguiente;
48     }
49     if (actual->siguiente != nullptr) {
50         Nodo* nodoABorrar = actual->siguiente;
51         actual->siguiente = nodoABorrar->siguiente;
52         delete nodoABorrar;
53         tamActual--;
54         return true;
55     }
56     return false;
57 }
58 void ListaPersonas::mostrar() {
59     if (cabeza == nullptr) {
60         cout << "La lista esta vacia." << endl;
61         return;
62     }
63     Nodo* actual = cabeza;
64     cout << "Lista de Contactos:\n";
65     while(actual != nullptr) {
66         cout << "[Nombre: " << actual->dato.nombre << " | Edad: " << actual->dato.edad << "] -> ";
67         actual = actual->siguiente;
68     }
69     cout << "NULL\n" << endl;
70 }
71 bool ListaPersonas::estaVacía() {
72     return cabeza == nullptr;
73 }
74 int ListaPersonas::getTamActual() {
75     return tamActual;
76 }
```

ListaPersonas.h

```
1  #ifndef LISTAPERSONAS_H
2  #define LISTAPERSONAS_H
3  #include <iostream>
4  #include <string>
5  using namespace std;
6  struct Persona {
7      string nombre;
8      int edad;
9  };
10 class Nodo {
11 public:
12     Persona dato;
13     Nodo* siguiente;
14 };
15 class ListaPersonas {
16 private:
17     Nodo* cabeza;
18     int tamActual;
19 public:
20     ListaPersonas();
21     ~ListaPersonas();
22
23     bool insertarOrdenado(string nom, int ed);
24     bool eliminarPorNombre(string nom);
25     void mostrar();
26     bool estaVacia();
27     int getTamActual();
28 };
29 #endif
```

Consola

```
--- Persona 2 ---
Nombre (sin espacios): daniel
Edad: 18
Se registro correctamente a la persona

<=== Menu ===>
1. Agregar persona
2. Eliminar persona por nombre
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir
Elige una opcion: 3

Mostrando lista...
Lista de Contactos:
[Nombre: daniel | Edad: 18] -> [Nombre: pepe | Edad: 19] -> NULL

<=== Menu ===>
1. Agregar persona
2. Eliminar persona por nombre
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir
Elige una opcion: 2

===== ELIMINAR POR NOMBRE =====
Que nombre deseas eliminar? (Respeta mayusculas)daniel
El registro de daniel ha sido eliminado.

<=== Menu ===>
1. Agregar persona
2. Eliminar persona por nombre
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir
Elige una opcion: 3

Mostrando lista...
Lista de Contactos:
[Nombre: pepe | Edad: 19] -> NULL
```


Se presentaron algunos problemas los cuales fueron:

El "efecto LIFO"

- *El problema:* Al probar las primeras inserciones, notamos que los datos se imprimían en orden inverso al que fueron ingresados (el último en entrar era el primero en mostrarse), a diferencia de la lista en Java donde mantenían el orden.

La solución y Análisis: Se comprendió que esto no era un error de C++, sino una decisión del algoritmo de inserción. Por eficiencia, nuestro método insertar() colocaba el nuevo Nodo al inicio de la lista (desplazando a la cabeza), lo que le da un comportamiento natural de Pila (LIFO). Si deseábamos un comportamiento estricto de lista cronológica, debíamos crear un puntero auxiliar que recorriera la lista hasta el final (nullptr) para enganchar ahí el nuevo Nodo. Esto nos enseñó cómo el manejo manual de punteros nos da control total sobre la topología de la estructura.

Fugas de Memoria y la necesidad del Destructor:

- *El problema:* Originalmente, nuestro programa terminaba su ejecución, pero los Nodos creados con new se quedaban atrapados en la memoria RAM porque no se destruían automáticamente como en Java (que cuenta con *Garbage Collector*).

La solución: Entendimos la regla de oro de C++: "Todo lo que se crea con new, se destruye con delete". Implementamos un método eliminar() manual y, más importante aún, codificamos el **Destructor** (~ListaDinamica()). El destructor utiliza un ciclo while para barrer y hacer delete de cada Nodo automáticamente en el último milisegundo antes de que el programa se cierre, garantizando una liberación de memoria impecable.

9. Librerías

El objetivo de este último programa es implementar el manejo de una Lista Dinámica aprovechando las herramientas nativas del lenguaje Java (específicamente la interfaz List y su implementación ArrayList o LinkedList de la librería java.util). Habiendo dominado previamente el manejo de memoria estática (arreglos manuales) y dinámica (punteros en C++), este ejercicio nos permite hacer la transición hacia el paradigma de programación declarativa. Aquí, la gestión del tamaño de la memoria, los desplazamientos (shifts) y los enlaces de los nodos son manejados internamente por la máquina virtual de Java, permitiéndonos enfocarnos únicamente en la lógica de negocio. Al igual que los anteriores, se hizo dos programas. **Dato básico** y **Nuevo tipo de dato**

Persona.java: Se mantiene nuestra clase base para definir el Nuevo Tipo de Dato con sus atributos y su método toString().

Main.java: La clase principal y administradora. Ya no requerimos programar una clase "Lista" desde cero. Simplemente importamos java.util.ArrayList, instanciamos los objetos y utilizamos los métodos nativos de la API de Java para manipular los datos.

En esta implementación, la abstracción es la protagonista. Cuando invocamos el método .add(elemento), Java evalúa por debajo si necesita redimensionar el arreglo en la memoria RAM y realiza la copia de datos automáticamente en un tiempo $O(1)$ amortizado. Del mismo modo, al utilizar .remove(indice), la librería se encarga internamente de ejecutar el ciclo for necesario para desplazar todos los elementos a la izquierda y tapar el "hueco" que queda en la memoria, librándonos del riesgo de fragmentar la lista o provocar desbordamientos de índice.

Dato Básico (DB)

ListaLibreriaBasica.java

```
1  > import ...
4
5  public class ListaLibreriaBasica extends ADTLibDB { 1 usage
6
7      private List<Integer> elementos; 9 usages
8
9  > public ListaLibreriaBasica() { this.elementos = new ArrayList<>(); }
12
13  @Override 1 usage
14  Ⓢ public void mostrar() {
15      if (elementos.isEmpty()) {
16          System.out.println("Lista vacia.");
17          return;
18      }
19      System.out.print("Lista: ");
20      for (Integer num : elementos) {
21          System.out.print(num + " ");
22      }
23      System.out.println();
24  }
25
26  @Override 1 usage
27  Ⓢ public boolean insertarOrdenado(int numero) {
28      elementos.add(numero);
29      Collections.sort(elementos);
30      return true;
31  }
32
33
34  Ⓢ public boolean eliminarPorValor(int numero) {
35      if (elementos.isEmpty()) return false;
36      return elementos.remove(Integer.valueOf(numero));
37  }
38
39  @Override 2 usages
40  Ⓢ public boolean estaVacia() {
41      if (getTamActual() == 0) {
42          System.out.println("Esta vacia la lista.");
43      } else {
44          System.out.println("No esta vacia tiene guardados: " + getTamActual());
45      }
46      return elementos.isEmpty();
47  }
48
49  @Override 2 usages
50  Ⓢ > public int getTamActual() { return elementos.size(); }
53 }
```

ADTLibDB.java

```
1  public abstract class ADTLibDB { 2 usages 1 inheritor
2      public abstract boolean insertarOrdenado(int numero);
3      public abstract boolean eliminarPorValor(int numero);
4      public abstract void mostrar(); 1 usage 1 implementation
5      public abstract boolean estaVacia(); 2 usages 1 implement
6      public abstract int getTamActual(); 2 usages 1 implementat
7  }
```

Consola

```
5. Salir
3

Mostrando lista...
Lista: 12 13
<===Menu===>
1. Agregar elemento
2. Eliminar elemento
3. Mostrar contendio
4. Esta la lista vacia?
5. Salir
2
No esta vacia tiene guardados: 2

===== ELIMINAR POR VALOR =====
Que numero deseas eliminar de la lista? 12
El numero 12 ha sido eliminado
<===Menu===>
1. Agregar elemento
2. Eliminar elemento
3. Mostrar contendio
4. Esta la lista vacia?
5. Salir
3

Mostrando lista...
Lista: 13
```

Nuevo Tipo de Dato

ListaLibreria.java

```
1  > import ...
3
4  public class ListaLibreria extends ADTLibNTD { 1 usage
5      private List<Persona> elementos; 11 usages
6      public ListaLibreria() { 1 usage
7          this.elementos = new ArrayList<>();
8      }
9      @Override 1 usage
10  ④ public void mostrar() {
11      if (elementos.isEmpty()) {
12          System.out.println("Lista vacia.");
13          return;
14      }
15      System.out.println("Lista:");
16      for (Persona p : elementos) {
17          System.out.println(p.toString());
18      }
19      System.out.println();
20  }
21  @Override 1 usage
22  ④ public boolean insertarOrdenado(Persona p) {
23      elementos.add(p);
24      elementos.sort((Persona p1, Persona p2) -> Integer.compare(p1.edad, p2.edad));
25      return true;
26  }
```

```
28  ④ public boolean eliminarPorNombre(String nombre) {
29      if (elementos.isEmpty()) return false;
30      for (int i = 0; i < elementos.size(); i++) {
31          if (elementos.get(i).nombre.equalsIgnoreCase(nombre)) {
32              elementos.remove(i);
33              return true;
34          }
35      }
36      return false;
37  }
38  @Override 1 usage
39  ④ public boolean estaVacia() {
40      if (getTamActual() == 0) {
41          System.out.println("Esta vacia la lista.");
42      } else {
43          System.out.println("No esta vacia tiene guardados: " + getTamActual());
44      }
45      return elementos.isEmpty();
46  }
47  @Override 3 usages
48  ④ > public int getTamActual() { return elementos.size(); }
51 }
```

ADTLibNTD.java

```
1  public abstract class ADTLibNTD { 2 usages 1 inheritor
2      public abstract boolean insertarOrdenado(Persona p); 1 usag
3      public abstract boolean eliminarPorNombre(String nombre);
4      public abstract void mostrar(); 1 usage 1 implementation
5      public abstract boolean estaVacia(); 1 usage 1 implementation
6      public abstract int getTamActual(); 3 usages 1 implementation
7  }
```

Consola

```
Lista:
Persona {Nombre: pepe Edad: 18 }
Persona {Nombre: daniel Edad: 19 }

<===Menu===>
1. Agregar elemento
2. Eliminar elemento
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir
2

===== ELIMINAR POR NOMBRE =====
Que nombre deseas eliminar de la lista? pepe
El registro de pepe ha sido eliminado
<===Menu===>
1. Agregar elemento
2. Eliminar elemento
3. Mostrar contenido
4. Esta la lista vacia?
5. Salir
3

Mostrando lista...
Lista:
Persona {Nombre: daniel Edad: 19 }
```

Se presenta un problema, veremos la solución y un aprendizaje de lo que se vio en arreglos, punteros y librerías

El conflicto de los Tipos Primitivos (Dato Básico) en las Colecciones:

- *El problema:* Al intentar crear la lista de datos básicos utilizando la sintaxis `ArrayList<int>` `listaNumeros = new ArrayList<>()`; el IDE de Java nos marcó un error de sintaxis que no teníamos al usar arreglos normales (`int[]`).

La solución: Comprendimos que las colecciones genéricas en Java (las que usan los picos `< >`) exigen que todo lo que guarden sea un "Objeto", no un dato primitivo suelto. Para solucionarlo, tuvimos que usar las "Clases Envoltorio" nativas de Java. Cambiamos la declaración a `ArrayList<Integer>`, lo que nos enseñó cómo el lenguaje "envuelve" un número básico en una caja orientada a objetos para poder administrarlo dinámicamente.

El salto del Paradigma Imperativo al Declarativo:

- *Aprendizaje teórico:* La mayor lección de este ejercicio fue comprender la diferencia entre enfoques. En los programas anteriores (arreglos y punteros), programamos de manera *imperativa*, controlando el "cómo" paso a paso (manejando punteros temporales, índices y destructores). Al usar la librería de Java, nuestro código se volvió *declarativo*; solo le indicamos a la computadora el "qué" queremos hacer (`listaNumeros.add()`), delegando la gestión de la máquina. Entender cuándo aplicar la micro-gestión (imperativo) y cuándo acelerar el desarrollo con APIs (declarativo).